arXiv:2307.08691v1 [cs.LG] 17 Jul 2023

FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning

Tri Dao[1,2]

[1]Department of Computer Science, Princeton University
[2]Department of Computer Science, Stanford University
trid@cs.stanford.edu

July 18, 2023

Abstract

Scaling Transformers to longer sequence lengths has been a major problem in the last several years, promising to improve performance in language modeling and high-resolution image understanding, as well as to unlock new applications in code, audio, and video generation. The attention layer is the main bottleneck in scaling to longer sequences, as its runtime and memory increase quadratically in the sequence length. FlashAttention [5] exploits the asymmetric GPU memory hierarchy to bring significant memory saving (linear instead of quadratic) and runtime speedup (2-4× compared to optimized baselines), with no approximation. However, FlashAttention is still not nearly as fast as optimized matrix-multiply (GEMM) operations, reaching only 25-40% of the theoretical maximum FLOPs/s. We observe that the inefficiency is due to suboptimal work partitioning between different thread blocks and warps on the GPU, causing either low-occupancy or unnecessary shared memory reads/writes. We propose FlashAttention-2, with better work partitioning to address these issues. In particular, we (1) tweak the algorithm to reduce the number of non-matmul FLOPs (2) parallelize the attention computation, even for a single head, across different thread blocks to increase occupancy, and (3) within each thread block, distribute the work between warps to reduce communication through shared memory. These yield around 2× speedup compared to FlashAttention, reaching 50-73% of the theoretical maximum FLOPs/s on A100 and getting close to the efficiency of GEMM operations. We empirically validate that when used end-to-end to train GPT-style models, FlashAttention-2 reaches training speed of up to 225 TFLOPs/s per A100 GPU (72% model FLOPs utilization).[1]

1 Introduction

Scaling up the context length of Transformers [18] is a challenge, since the attention layer at their heart has runtime and memory requirements quadratic in the input sequence length. Ideally, we would like to go beyond the standard 2k sequence length limit to train models to understand books, high resolution images, and long-form videos. Just within the last year, there have been several language models with much longer context than before: GPT-4 [12] with context length 32k, MosaicML's MPT with context length 65k, and Anthropic's Claude with context length 100k. Emerging use cases such as long document querying and story writing have demonstrated a need for models with such long context.

To reduce the computational requirement of attention on such long context, there have been numerous methods proposed to approximate attention [2, 3, 4, 8, 9, 14, 19, 20]. Though these methods have seen some use cases, as far as we know, most large-scale training runs still use standard attention. Motivated by this, Dao et al. [5] proposed to reorder the attention computation and leverages classical techniques (tiling, recomputation) to significantly speed it up and reduce memory usage from quadratic to linear in sequence length. This yields 2-4× wall-clock time speedup over optimized baselines, up to 10-20× memory saving,

[1]FlashAttention-2 is available at https://github.com/Dao-AILab/flash-attention

arXiv:2307.08691v1 [cs.LG] 17 Jul 2023

FlashAttention-2版本：具备更优并行性及任务分区的更快注意力机制

Tri Dao[1,2]

[1]普林斯顿大学计算机科学系[2]斯坦福大学计算机科学系
trid@cs.stanford.edu

2023年7月18日

摘要

Scaling Transformers to longer sequence lengths has been a major problem in the last several years, promising to improve performance in language modeling and high-resolution image understanding, as well as to unlock new applications in code, audio, and video generation. The attention layer is the main bottleneck in scaling to longer sequences, as its runtime and memory increase quadratically in the sequence length. FlashAttention [5] exploits the asymmetric GPU memory hierarchy to bring significant memory saving (linear instead of quadratic) and runtime speedup (2-4× compared to optimized baselines), with no approximation. However, FlashAttention is still not nearly as fast as optimized matrix-multiply (GEMM) operations, reaching only 25-40% of the theoretical maximum FLOPs/s. We observe that the inefficiency is due to suboptimal work partitioning between different thread blocks and warps on the GPU, causing either low-occupancy or unnecessary shared memory reads/writes. We propose FlashAttention-2, with better work partitioning to address these issues. In particular, we (1) tweak the algorithm to reduce the number of non-matmul FLOPs (2) parallelize the attention computation, even for a single head, across different thread blocks to increase occupancy, and (3) within each thread block, distribute the work between warps to reduce communication through shared memory. These yield around 2× speedup compared to FlashAttention, reaching 50-73% of the theoretical maximum FLOPs/s on A100 and getting close to the efficiency of GEMM operations. We empirically validate that when used end-to-end to train GPT-style models, FlashAttention-2 reaches training speed of up to 225 TFLOPs/s per A100 GPU (72% model FLOPs utilization).[1]

1 引言

提升Transformers模型的序列长度 [18] 是一项颇具挑战性的任务，因为其核心的注意力机制对运行时间和内存的需求与输入序列长度呈二次方关系。理想情况下，我们希望突破现有的2千序列长度限制，让模型能够理解书籍、高分辨率图像以及长时长视频内容。仅在过去一年里，就已经出现了多款具备更长上下文处理能力的语言模型：上下文长度达32千的GPT-4 [12] ，上下文长度为65千的MosaicML推出的MPT模型，以及上下文长度可达100千的Anthropic开发的Claude模型。诸如长文档查询和故事创作等新兴应用场景，也进一步凸显了对这类具备超长上下文处理能力的模型的需求。

为降低此类长上下文场景下注意力机制的计算开销，已有众多方法被提出用于对注意力机制进行近似计算 [2, 3, 4, 8, 9, 14, 19, 20]。尽管这些方法在某些应用场景中得到应用，但据我们所知，大多数大规模训练仍采用标准的注意力机制。受此启发，Dao等人 [5] 提出了重新排序注意力计算顺序的方法，并借助平铺处理、重新计算等传统技术，大幅提升了计算速度，同时将内存使用量从与序列长度成二次方关系降至线性关系。这样一来，相较于经过优化的基准方案，其加速比可达2-4× 倍，内存占用则可减少多达10-20× 。

[1]FlashAttention-2的代码地址为https://github.com/Dao-AILab/flash-attention

with no approximation, and as a result FlashAttention has seen wide adoption in large-scale training and inference of Transformers.

However, context length increases even more, FlashAttention is still not nearly as efficient as other primitives such as matrix-multiply (GEMM). In particular, while FlashAttention is already 2-4× faster than a standard attention implementation, the forward pass only reaches 30-50% of the theoretical maximum FLOPs/s of the device (Fig. 5), while the backward pass is even more challenging, reaching only 25-35% of maximum throughput on A100 GPU (Fig. 6). In contrast, optimized GEMM can reach up to 80-90% of the theoretical maximum device throughput. Through careful profiling, we observe that FlashAttention still has suboptimal work partitioning between different thread blocks and warps on the GPU, causing either low-occupancy or unnecessary shared memory reads/writes.

Building on FlashAttention, we propose FlashAttention-2 with better parallelism and work partitioning to address these challenges.

1. In Section 3.1, we tweak the algorithms to reduce the number of non-matmul FLOPs while not changing the output. While the non-matmul FLOPs only account for a small fraction of the total FLOPs, they take longer to perform as GPUs have specialized units for matrix multiply, and as a result the matmul throughput can be up to 16× higher than non-matmul throughput. It is thus important to reduce non-matmul FLOPs and spend as much time as possible doing matmul FLOPs.

2. We propose to parallelize both the forward pass and backward pass along the sequence length dimension, in addition to the batch and number of heads dimension. This increases occupancy (utilization of GPU resources) in the case where the sequences are long (and hence batch size is often small).

3. Even within one block of attention computation, we partition the work between different warps of a thread block to reduce communication and shared memory reads/writes.

In Section 4, we empirically validate that FlashAttention-2 yields significant speedup compared to even FlashAttention. Benchmarks on different settings (with or without causal mask, different head dimensions) show that FlashAttention-2 achieves around 2× speedup over FlashAttention, reaching up to 73% of the theoretical max throughput in the forward pass, and up to 63% of the theoretical max throughput in the backward pass. When used end-to-end to train GPT-style models, we reach training speed of up to 225 TFLOPs/s per A100 GPU.

2 Background

We provide some background on the performance characteristics and execution model of GPUs. We also describe the standard implementation of attention, as well as FlashAttention.

2.1 Hardware characteristics

GPU performance characteristics. The GPU consists of compute elements (e.g., floating point arithmetic units) and a memory hierarchy. Most modern GPUs contain specialized units to accelerate matrix multiply in low-precision (e.g., Tensor Cores on Nvidia GPUs for FP16/BF16 matrix multiply). The memory hierarchy comprise of high bandwidth memory (HBM), and on-chip SRAM (aka shared memory). As an example, the A100 GPU has 40-80GB of high bandwidth memory (HBM) with bandwidth 1.5-2.0TB/s and 192KB of on-chip SRAM per each of 108 streaming multiprocessors with bandwidth estimated around 19TB/s [6, 7]. As the L2 cache is not directly controllable by the programmer, we focus on the HBM and SRAM for the purpose of this discussion.

Execution Model. GPUs have a massive number of threads to execute an operation (called a kernel). Threads are organized into thread blocks, which are scheduled to run on streaming multiprocessors (SMs). Within each thread blocks, threads are grouped into warps (a group of 32 threads). Threads within a warp can communicate by fast shuffle instructions or cooperate to perform matrix multiply. Warps within a thread block can communicate by reading from / writing to shared memory. Each kernel loads inputs from HBM to registers and SRAM, computes, then writes outputs to HBM.

由于无需进行任何近似处理，FlashAttention算法已在Transformers模型的大规模训练与推理任务中得到了广泛的应用。

然而，即便上下文长度进一步增加，FlashAttention的效率依然远不及矩阵乘法（GEMM）等其他基础运算方式。具体而言，虽然FlashAttention的速度已比标准注意力机制实现快2-4× 倍，但其前向传播时的每秒浮点运算次数仅能达到该设备理论最大值的30%-50%（见图5）；而后向传播的难度更大，在A100显卡上的最高处理效率仅能达到理论最大值的25%-35%（见图6）。相比之下，经过优化的GEMM运算甚至能实现接近设备理论最大处理效率的80%-90%。通过仔细的性能分析，我们发现FlashAttention在GPU的不同线程块和线程组之间仍存在不理想的任务分配方式，从而导致资源利用率低下或出现不必要的共享内存读写操作。

在FlashAttention算法的基础上，我们提出了具有更优并行性及任务划分能力的FlashAttention-2版本，以此应对这些挑战。

1. 在3.1节中，我们对算法进行了优化，在不改变输出结果的前提下减少了非矩阵乘法类型的浮点运算次数。虽然非矩阵乘法类型的浮点运算次数在总浮点运算次数中所占比例很小，但由于图形处理器拥有专用于矩阵乘法的专用单元，执行这类操作需要更长的时间；因此，矩阵乘法的处理速度可比非矩阵乘法类型高出 16× 倍。所以，减少非矩阵乘法类型的浮点运算次数，并尽可能多地将时间用于执行矩阵乘法操作，显得尤为重要。

2. 除了在批次大小和注意力头数维度上进行并行处理外，我们还提议在序列长度维度上同时对前向传播和后向传播进行并行化。这样一来，在序列较长（因而批次大小通常较小）的情况下，能够进一步提升图形处理器资源的利用率。

3. 即便在同一个注意力计算块内部，我们也会将任务分配给线程块中的不同线程组，以此减少通信开销以及共享内存的读写操作。

在第4节中，我们通过实验证明，即便与FlashAttention相比，FlashAttention-2也能带来显著的加速比。在不同配置下的测试（包含与不包含因果掩码、不同头维度）显示，FlashAttention-2相较于FlashAttention的加速比可达 2× 倍，在前向传播时的理论最大吞吐量利用率可达到73%，而在后向传播时则可达63%。当将其端到端用于训练GPT风格模型时，每块A100显卡的训练速度最高可达到225每秒万亿次浮点运算次数。

2 背景

我们提供一些示例。我们还会阐述注意力机制以及FlashAttention算法的标准化实现方式。
描述标准化的实现方式。注意力机制的实现方式，以及FlashAttention算法的实现。

2.1 硬件特性

图形处理器的性能特性。图形处理器由计算单元（例如浮点运算单元）以及内存层次结构组成。大多数现代图形处理器都配备了专门用于加速低精度矩阵乘法的单元（比如Nvidia图形处理器上的张量核心，可用于FP16/BF16格式的矩阵乘法）。其内存层次结构包括高带宽内存（HBM）以及片上SRAM（亦即共享内存）。以A100显卡为例，它拥有40-80GB容量的高带宽内存（HBM），带宽可达1.5-2.0TB/s；同时，它的108个流式多处理器每个都配备192KB的片上SRAM，这些SRAM的总带宽约为19TB/s [6, 7]。由于L2缓存无法由程序员直接控制，因此在本次讨论中我们将重点关注HBM和SRAM。

执行模型。图形处理器拥有海量线程来执行某项操作（即内核）。这些线程被组织成线程块，随后由流式多处理器负责调度其运行。在每个线程块内部，线程们又被进一步划分为线程组（由32个线程组成）。同一线程组内的线程可通过快速的洗牌指令进行通信，或协作完成矩阵乘法运算。而线程块内的不同线程组则可以通过读写共享内存来实现信息交互。每个内核都会先将输入数据从高带宽内存加载到寄存器和片上静态随机存取存储器中进行处理，最后再将结果写入高带宽内存。

2.2 Standard Attention Implementation

Given input sequences $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ where N is the sequence length and d is the head dimension, we want to compute the attention output $\mathbf{O} \in \mathbb{R}^{N \times d}$:

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d},$$

where softmax is applied row-wise.[2] For multi-head attention (MHA), this same computation is performed in parallel across many heads, and parallel over the batch dimension (number of input sequences in a batch).

The backward pass of attention proceeds as follows. Let $\mathbf{dO} \in \mathbb{R}^{N \times d}$ be the gradient of $\mathbf{O}$ with respect to some loss function. Then by the chain rule (aka backpropagation):

$$\begin{aligned}
\mathbf{dV} &= \mathbf{P}^\top \mathbf{dO} \in \mathbb{R}^{N \times d} \\
\mathbf{dP} &= \mathbf{dO}\mathbf{V}^\top \in \mathbb{R}^{N \times N} \\
\mathbf{dS} &= \mathrm{dsoftmax}(\mathbf{dP}) \in \mathbb{R}^{N \times N} \\
\mathbf{dQ} &= \mathbf{dS}\mathbf{K} \in \mathbb{R}^{N \times d} \\
\mathbf{dK} &= \mathbf{Q}\mathbf{dS}^\top \in \mathbb{R}^{N \times d},
\end{aligned}$$

where dsoftmax is the gradient (backward pass) of softmax applied row-wise. One can work out that if $p = \mathrm{softmax}(s)$ for some vector s and p, then with output gradient dp, the input gradient $ds = (\mathrm{diag}(p) - pp^\top)dp$.

Standard attention implementations materialize the matrices $\mathbf{S}$ and $\mathbf{P}$ to HBM, which takes $O(N^2)$ memory. Often $N \gg d$ (typically N is on the order of 1k–8k and d is around 64–128). The standard attention implementation (1) calls the matrix multiply (GEMM) subroutine to multiply $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, writes the result to HBM, then (2) loads § from HBM to compute softmax and write the result $\mathbf{P}$ to HBM, and finally (3) calls GEMM to get $\mathbf{O} = \mathbf{P}\mathbf{V}$. As most of the operations are bounded by memory bandwidth, the large number of memory accesses translates to slow wall-clock time. Moreover, the required memory is $O(N^2)$ due to having to materialize $\mathbf{S}$ and $\mathbf{P}$. Moreover, one has to save $\mathbf{P} \in \mathbb{R}^{N \times N}$ for the backward pass to compute the gradients.

2.3 FlashAttention

To speed up attention on hardware accelerators such as GPU, [5] proposes an algorithm to reduce the memory reads/writes while maintaining the same output (without approximation).

2.3.1 Forward pass

FlashAttention applies the classical technique of tiling to reduce memory IOs, by (1) loading blocks of inputs from HBM to SRAM, (2) computing attention with respect to that block, and then (3) updating the output without writing the large intermediate matrices $\mathbf{S}$ and $\mathbf{P}$ to HBM. As the softmax couples entire rows or blocks of row, online softmax [11, 13] can split the attention computation into blocks, and rescale the output of each block to finally get the right result (with no approximation). By significantly reducing the amount of memory reads/writes, FlashAttention yields 2-4× wall-clock speedup over optimized baseline attention implementations.

We describe the online softmax technique [11] and how it is used in attention [13]. For simplicity, consider just one row block of the attention matrix $\mathbf{S}$, of the form $\begin{bmatrix}\mathbf{S}^{(1)} & \mathbf{S}^{(2)}\end{bmatrix}$ for some matrices $\mathbf{S}^{(1)}, \mathbf{S}^{(2)} \in \mathbb{R}^{B_r \times B_c}$, where B_r and B_c are the row and column block sizes. We want to compute softmax of this row block and multiply with the value, of the form $\begin{bmatrix}\mathbf{V}^{(1)} \\ \mathbf{V}^{(2)}\end{bmatrix}$ for some matrices $\mathbf{V}^{(1)}, \mathbf{V}^{(2)} \in \mathbb{R}^{B_c \times d}$. Standard softmax would

[2] For clarity of exposition, we omit the scaling of $\mathbf{Q}\mathbf{K}^\top$ (typically by 1/d), and optionally elementwise masking on $\mathbf{S}$ and/or dropout applied to $\mathbf{P}$

2.2 标准注意力机制的实现

给定输入序列 $\mathbf{Q}$、$\mathbf{K}$以及$\mathbf{V} \in \mathbb{R}^{N \times d}$，其中 N表示序列长度， d表示头维度，我们需要计算注意力机制的输出 $\mathbf{O} \in \mathbb{R}^{N \times d}$:

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d},$$

其中softmax函数是按行应用的。[2] 对于多头注意力机制（MHA），相同的计算会在多个注意力头之间并行执行，同时也会在批次维度（即一个批次中的输入序列数量）上实现并行处理。

注意力机制的后向传播流程如下。设$\mathbf{dO} \in \mathbb{R}^{N \times d}$ 为$\mathbf{O}$ 相对于某个损失函数的梯度。根据链式法则（亦即反向传播）：

$$\begin{aligned}
\mathbf{dV} &= \mathbf{P}^\top \mathbf{dO} \in \mathbb{R}^{N \times d} \\
\mathbf{dP} &= \mathbf{dO}\mathbf{V}^\top \in \mathbb{R}^{N \times N} \\
\mathbf{dS} &= \mathrm{dsoftmax}(\mathbf{dP}) \in \mathbb{R}^{N \times N} \\
\mathbf{dQ} &= \mathbf{dS}\mathbf{K} \in \mathbb{R}^{N \times d} \\
\mathbf{dK} &= \mathbf{Q}\mathbf{dS}^\top \in \mathbb{R}^{N \times d},
\end{aligned}$$

其中dsoftmax表示按行应用的softmax函数的梯度（即后向传播结果）。通过推导可以得知，对于某些向量 s和 p，若其softmax运算结果为 p =softmax(s)，则在输出梯度为 dp的情况下，输入梯度为 ds =(diag(p) $- pp^\top)dp$。

Standard attention implementations materialize the matrices $\mathbf{S}$ and $\mathbf{P}$ to HBM, which takes $O(N^2)$ memory. Often $N \gg d$ (typically N is on the order of 1k–8k and d is around 64–128). The standard attention implementation (1) calls the matrix multiply (GEMM) subroutine to multiply $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, writes the result to HBM, then (2) loads § from HBM to compute softmax and write the result $\mathbf{P}$ to HBM, and finally (3) calls GEMM to get $\mathbf{O} = \mathbf{P}\mathbf{V}$. As most of the operations are bounded by memory bandwidth, the large number of memory accesses translates to slow wall-clock time. Moreover, the required memory is $O(N^2)$ due to having to materialize $\mathbf{S}$ and $\mathbf{P}$. Moreover, one has to save $\mathbf{P} \in \mathbb{R}^{N \times N}$ for the backward pass to compute the gradients.

2.3 FlashAttention算法

为在图形处理器等硬件加速器上提升注意力机制的运算速度， [5]提出了一种算法，可在保持相同输出结果（无需近似处理）的同时减少内存读写操作。

2.3.1 前向传播

FlashAttention算法运用经典的平铺处理技术来减少内存IO操作，具体流程为：首先从高带宽内存中将输入数据块加载到片上静态随机存取存储器中，接着针对该数据块计算注意力机制，最后在无需将庞大的中间矩阵$\mathbf{S}$ 和$\mathbf{P}$ 写回高带宽内存的情况下更新输出结果。由于softmax函数需要处理整行数据或整行数据块，因此在线soft max [11, 13] 可以将注意力计算拆分为多个小块，并对每个块的输出结果进行重新缩放，从而最终得到精确的结果（无需任何近似处理）。通过大幅降低内存读写操作的频率，FlashAttention算法相比优化后的基准注意力实现，能够实现2-4× 倍的实时时钟加速比。

We describe the online softmax technique [11] and how it is used in attention [13]. For simplicity, consider just one row block of the attention matrix $\mathbf{S}$, of the form $\begin{bmatrix}\mathbf{S}^{(1)} & \mathbf{S}^{(2)}\end{bmatrix}$ for some matrices $\mathbf{S}^{(1)}, \mathbf{S}^{(2)} \in \mathbb{R}^{B_r \times B_c}$, where B_r and B_c are the row and column block sizes. We want to compute softmax of this row block and multiply with the value, of the form $\begin{bmatrix}\mathbf{V}^{(1)} \\ \mathbf{V}^{(2)}\end{bmatrix}$ for some matrices $\mathbf{V}^{(1)}, \mathbf{V}^{(2)} \in \mathbb{R}^{B_c \times d}$. Standard softmax would

[2] 为便于阐述，我们省略了$\mathbf{Q}\mathbf{K}^\top$ 的缩放操作（通常为1/d），同时也可选地不进行$\mathbf{S}$ 的逐元素掩码处理，以及不对$\mathbf{P}$应用丢弃机制。

compute:

$$
\begin{aligned}
m &= \max(\text{rowmax}(\mathbf{S}^{(1)}), \text{rowmax}(\mathbf{S}^{(2)})) \in \mathbb{R}^{B_r} \\
\ell &= \text{rowsum}(e^{\mathbf{S}^{(1)}-m}) + \text{rowsum}(e^{\mathbf{S}^{(2)}-m}) \in \mathbb{R}^{B_r} \\
\mathbf{P} &= \begin{bmatrix}\mathbf{P}^{(1)} & \mathbf{P}^{(2)}\end{bmatrix} = \text{diag}(\ell)^{-1}\begin{bmatrix}e^{\mathbf{S}^{(1)}-m} & e^{\mathbf{S}^{(2)}-m}\end{bmatrix} \in \mathbb{R}^{B_r \times 2B_c} \\
\mathbf{O} &= \begin{bmatrix}\mathbf{P}^{(1)} & \mathbf{P}^{(2)}\end{bmatrix}\begin{bmatrix}\mathbf{V}^{(1)} \\ \mathbf{V}^{(2)}\end{bmatrix} = \text{diag}(\ell)^{-1}e^{\mathbf{S}^{(1)}-m}\mathbf{V}^{(1)} + e^{\mathbf{S}^{(2)}-m}\mathbf{V}^{(2)} \in \mathbb{R}^{B_r \times d}.
\end{aligned}
$$

Online softmax instead computes "local" softmax with respect to each block and rescale to get the right output at the end:

$$
\begin{aligned}
m^{(1)} &= \text{rowmax}(\mathbf{S}^{(1)}) \in \mathbb{R}^{B_r} \\
\ell^{(1)} &= \text{rowsum}(e^{\mathbf{S}^{(1)}-m^{(1)}}) \in \mathbb{R}^{B_r} \\
\tilde{\mathbf{P}}^{(1)} &= \text{diag}(\ell^{(1)})^{-1}e^{\mathbf{S}^{(1)}-m^{(1)}} \in \mathbb{R}^{B_r \times B_c} \\
\mathbf{O}^{(1)} &= \tilde{\mathbf{P}}^{(1)}\mathbf{V}^{(1)} = \text{diag}(\ell^{(1)})^{-1}e^{\mathbf{S}^{(1)}-m^{(1)}}\mathbf{V}^{(1)} \in \mathbb{R}^{B_r \times d} \\
m^{(2)} &= \max(m^{(1)}, \text{rowmax}(\mathbf{S}^{(2)})) = m \\
\ell^{(2)} &= e^{m^{(1)}-m^{(2)}}\ell^{(1)} + \text{rowsum}(e^{\mathbf{S}^{(2)}-m^{(2)}}) = \text{rowsum}(e^{\mathbf{S}^{(1)}-m}) + \text{rowsum}(e^{\mathbf{S}^{(2)}-m}) = \ell \\
\tilde{\mathbf{P}}^{(2)} &= \text{diag}(\ell^{(2)})^{-1}e^{\mathbf{S}^{(2)}-m^{(2)}} \\
\mathbf{O}^{(2)} &= \text{diag}(\ell^{(1)}/\ell^{(2)})^{-1}\mathbf{O}^{(1)} + \tilde{\mathbf{P}}^{(2)}\mathbf{V}^{(2)} = \text{diag}(\ell^{(2)})^{-1}e^{s^{(1)}-m}\mathbf{V}^{(1)} + \text{diag}(\ell^{(2)})^{-1}e^{s^{(2)}-m}\mathbf{V}^{(2)} = \mathbf{O}.
\end{aligned}
$$

We show how FlashAttention uses online softmax to enable tiling (Fig. 1) to reduce memory reads/writes.

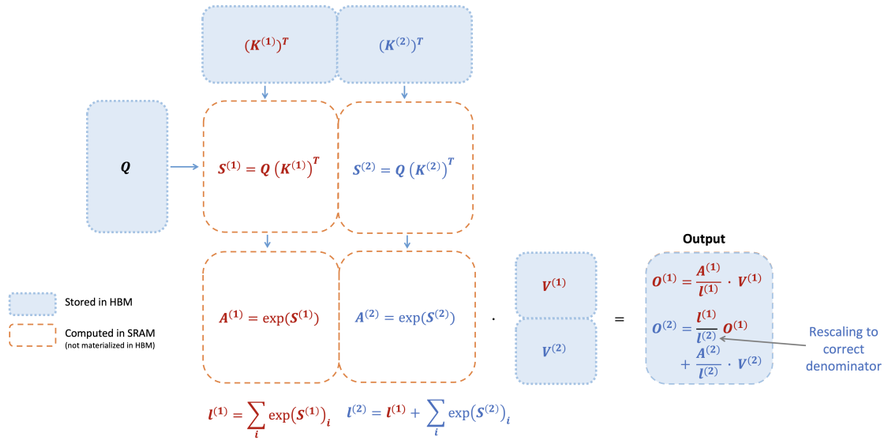


Figure 1: Diagram of how FlashAttention forward pass is performed, when the key $\mathbf{K}$ is partitioned into two blocks and the value $\mathbf{V}$ is also partitioned into two blocks. By computing attention with respect to each block and rescaling the output, we get the right answer at the end, while avoiding expensive memory reads/writes of the intermediate matrices $\mathbf{S}$ and $\mathbf{P}$. We simplify the diagram, omitting the step in softmax that subtracts each element by the row-wise max.

计算：

$$
\begin{aligned}
m &= \max(\text{rowmax}(\mathbf{S}^{(1)}), \text{rowmax}(\mathbf{S}^{(2)})) \in \mathbb{R}^{B_r} \\
\ell &= \text{rowsum}(e^{\mathbf{S}^{(1)}-m}) + \text{rowsum}(e^{\mathbf{S}^{(2)}-m}) \in \mathbb{R}^{B_r} \\
\mathbf{P} &= \begin{bmatrix}\mathbf{P}^{(1)} & \mathbf{P}^{(2)}\end{bmatrix} = \text{diag}(\ell)^{-1}\begin{bmatrix}e^{\mathbf{S}^{(1)}-m} & e^{\mathbf{S}^{(2)}-m}\end{bmatrix} \in \mathbb{R}^{B_r \times 2B_c} \\
\mathbf{O} &= \begin{bmatrix}\mathbf{P}^{(1)} & \mathbf{P}^{(2)}\end{bmatrix}\begin{bmatrix}\mathbf{V}^{(1)} \\ \mathbf{V}^{(2)}\end{bmatrix} = \text{diag}(\ell)^{-1}e^{\mathbf{S}^{(1)}-m}\mathbf{V}^{(1)} + e^{\mathbf{S}^{(2)}-m}\mathbf{V}^{(2)} \in \mathbb{R}^{B_r \times d}.
\end{aligned}
$$

在线softmax算法 具体而言，该算法会先对每个块独立执行注意力前向计算，随后对得到的输出进行重新缩放，从而得到正确的结果。
最终得到的输出即为所求的哈希值。

$$
\begin{aligned}
m^{(1)} &= \text{rowmax}(\mathbf{S}^{(1)}) \in \mathbb{R}^{B_r} \\
\ell^{(1)} &= \text{rowsum}(e^{\mathbf{S}^{(1)}-m^{(1)}}) \in \mathbb{R}^{B_r} \\
\tilde{\mathbf{P}}^{(1)} &= \text{diag}(\ell^{(1)})^{-1}e^{\mathbf{S}^{(1)}-m^{(1)}} \in \mathbb{R}^{B_r \times B_c} \\
\mathbf{O}^{(1)} &= \tilde{\mathbf{P}}^{(1)}\mathbf{V}^{(1)} = \text{diag}(\ell^{(1)})^{-1}e^{\mathbf{S}^{(1)}-m^{(1)}}\mathbf{V}^{(1)} \in \mathbb{R}^{B_r \times d} \\
m^{(2)} &= \max(m^{(1)}, \text{rowmax}(\mathbf{S}^{(2)})) = m \\
\ell^{(2)} &= e^{m^{(1)}-m^{(2)}}\ell^{(1)} + \text{rowsum}(e^{\mathbf{S}^{(2)}-m^{(2)}}) = \text{rowsum}(e^{\mathbf{S}^{(1)}-m}) + \text{rowsum}(e^{\mathbf{S}^{(2)}-m}) = \ell \\
\tilde{\mathbf{P}}^{(2)} &= \text{diag}(\ell^{(2)})^{-1}e^{\mathbf{S}^{(2)}-m^{(2)}} \\
\mathbf{O}^{(2)} &= \text{diag}(\ell^{(1)}/\ell^{(2)})^{-1}\mathbf{O}^{(1)} + \tilde{\mathbf{P}}^{(2)}\mathbf{V}^{(2)} = \text{diag}(\ell^{(2)})^{-1}e^{s^{(1)}-m}\mathbf{V}^{(1)} + \text{diag}(\ell^{(2)})^{-1}e^{s^{(2)}-m}\mathbf{V}^{(2)} = \mathbf{O}.
\end{aligned}
$$

我们展示了FI如何实现FlashAttention算法通过采用在线softmax函数实现平铺处理（见图1），从而减少内存读写操作。 .

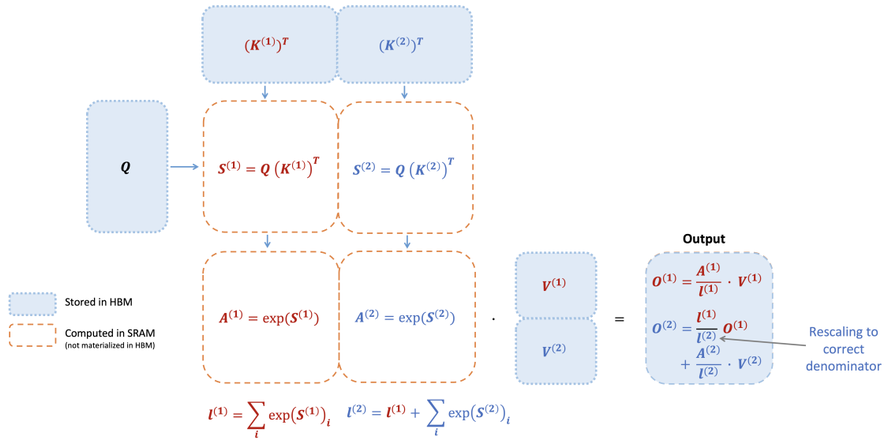


图1展示了当键$\mathbf{K}$ 和值$\mathbf{V}$ 都被分割成两个块时，FlashAttention算法的执行流程。通过针对每个块分别计算注意力机制并重新缩放输出，我们最终能够得到正确结果，同时避免了处理中间矩阵$\mathbf{S}$ 和$\mathbf{P}$时所需的昂贵内存读写操作。图中为了简化表述，省略了softmax函数中减去每行最大值的步骤。

2.3.2 Backward pass

In the backward pass, by re-computing the values of the attention matrices $\mathbf{S}$ and $\mathbf{P}$ once blocks of inputs $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ are already loaded to SRAM, FlashAttention avoids having to store large intermediate values. By not having to save the large matrices $\mathbf{S}$ and $\mathbf{P}$ of size $N \times N$, FlashAttention yields 10-20× memory saving depending on sequence length (memory required in linear in sequence length N instead of quadratic). The backward pass also achieves 2-4× wall-clock speedup due to reduce memory reads/writes.

The backward pass applies tiling to the equations in Section 2.2. Though the backward pass is simpler than the forward pass conceptually (there is no softmax rescaling), the implementation is significantly more involved. This is because there are more values to be kept in SRAM to perform 5 matrix multiples in the backward pass, compared to just 2 matrix multiples in the forward pass.

3 FlashAttention-2: Algorithm, Parallelism, and Work Partitioning

We describe the FlashAttention-2 algorithm, which includes several tweaks to FlashAttention to reduce the number of non-matmul FLOPs. We then describe how to parallelize the computation on different thread blocks to make full use the GPU resources. Finally we describe we partition the work between different warps within one thread block to reduce the amount of shared memory access. These improvements lead to 2-3× speedup as validated in Section 4.

3.1 Algorithm

We tweak the algorithm from FlashAttention to reduce the number of non-matmul FLOPs. This is because modern GPUs have specialized compute units (e.g., Tensor Cores on Nvidia GPUs) that makes matmul much faster. As an example, the A100 GPU has a max theoretical throughput of 312 TFLOPs/s of FP16/BF16 matmul, but only 19.5 TFLOPs/s of non-matmul FP32. Another way to think about this is that each non-matmul FLOP is 16× more expensive than a matmul FLOP. To maintain high throughput (e.g., more than 50% of the maximum theoretical TFLOPs/s), we want to spend as much time on matmul FLOPs as possible.

3.1.1 Forward pass

We revisit the online softmax trick as shown in Section 2.3 and make two minor tweaks to reduce non-matmul FLOPs:

1. We do not have to rescale both terms of the output update by $\mathrm{diag}(\ell^{(2)})^{-1}$:

$$\mathbf{O}^{(2)} = \mathrm{diag}(\ell^{(1)}/\ell^{(2)})^{-1}\mathbf{O}^{(1)} + \mathrm{diag}(\ell^{(2)})^{-1}e^{\mathbf{S}^{(2)}-m^{(2)}}\mathbf{V}^{(2)}.$$

   We can instead maintain an "un-scaled" version of $\mathbf{O}^{(2)}$ and keep around the statistics $\ell^{(2)}$:

$$\tilde{\mathbf{O}}^{(2)} = \mathrm{diag}(\ell^{(1)})^{-1}\mathbf{O}^{(1)} + e^{\mathbf{S}^{(2)}-m^{(2)}}\mathbf{V}^{(2)}.$$

   Only at the every end of the loop do we scale the final $\tilde{\mathbf{O}}^{(\mathrm{last})}$ by $\mathrm{diag}(\ell^{(\mathrm{last})})^{-1}$ to get the right output.

2. We do not have to save both the max $m^{(j)}$ and the sum of exponentials $\ell^{(j)}$ for the backward pass. We only need to store the logsumexp $L^{(j)} = m^{(j)} + \log(\ell^{(j)})$.

2.3.2 后向传播

在后向传播过程中，当输入块**Q**、**K**以及**V**已被加载到片上静态随机存取存储器中时，FlashAttention算法会重新计算注意力矩阵**S** 和**P** 的值。由于无需存储大小为 $N \times N$的庞大中间矩阵，该算法能够根据序列长度实现10-20× 的内存节省效果（所需内存与序列长度呈线性关系 N ，而非二次方关系）。此外，得益于内存读写操作的减少，后向传播的墙钟时间加速比也可达到2-4× 。

在后续的后向传播过程中，会对2.2节中的方程式进行平铺处理。虽然从概念上来看后向传播比前向传播更简单（无需通过softmax函数进行重新缩放），但其实现难度却要大得多。这是因为在后向传播中需要进行5次矩阵乘法运算，需要存储在片上静态随机存取存储器中的数值量远多于前向传播中的2次矩阵乘法运算所需的数值量。

3 FlashAttention-2：算法、并行性以及任务划分

我们介绍了FlashAttention-2算法，该算法在FlashAttention的基础上进行了一系列优化，旨在减少非矩阵乘法类的浮点运算次数。随后，我们阐述了如何在不同线程块上并行执行计算，从而充分利用图形处理器的资源。最后，我们介绍了如何在一个线程块内的不同线程组之间分配任务，以降低对共享内存的访问频率。正如第4节所验证的，这些改进带来了2-3×倍的加速比。

3.1 算法

我们对FlashAttention算法进行了优化，旨在减少非矩阵乘法类型的浮点运算次数。这是因为现代图形处理器配备了专门的计算单元（例如Nvidia显卡上的张量核心），这使得矩阵乘法运算速度大幅提升。以A100显卡为例，其FP16/BF16格式的矩阵乘法理论最高吞吐量可达312每秒万亿次浮点运算次数，而非矩阵乘法的FP32格式则仅有19.5每秒万亿次浮点运算次数。另一种理解方式是，每执行一次非矩阵乘法浮点运算所需的成本比矩阵乘法高 16× 倍。为保持较高的运算吞吐量（即超过理论最大值的50%），我们希望尽可能多地将时间用于矩阵乘法运算。

3.1.1 前向传播

我们重新采用了2.3节中介绍的在线softmax技巧，并对其进行了两项细微调整，以此降低非矩阵乘法相关的浮点运算次数。

首先，无需再通过对$\mathrm{diag}(l^{(2)})^{-1}$得到的输出更新项进行重新缩放来处理问题。

$$\mathbf{O}^{(2)} = \mathrm{diag}(\ell^{(1)}/\ell^{(2)})^{-1}\mathbf{O}^{(1)} + \mathrm{diag}(\ell^{(2)})^{-1}e^{\mathbf{S}^{(2)}-m^{(2)}}\mathbf{V}^{(2)}.$$

相反，我们可以保留$\mathbf{O}^{(2)}$ 的“未缩放”版本，同时继续使用$l^{(2)}$这类统计量。

$$\tilde{\mathbf{O}}^{(2)} = \mathrm{diag}(\ell^{(1)})^{-1}\mathbf{O}^{(1)} + e^{\mathbf{S}^{(2)}-m^{(2)}}\mathbf{V}^{(2)}.$$

仅在即将到来之际 在循环结束时，我们需要通过 $\mathrm{diag}(l^{(\mathrm{last})})^{-1}$ 对最终的 $\tilde{}\mathbf{O}^{(\mathrm{last})}$ 进行缩放，从而得到正确的输出结果。 t.

2. 在进行后向传播时，我们无需同时保存最大值 $m^{(j)}$以及指数之和 $l^{(j)}$。我们只需存储对数和指数运算的结果 $L_{(j)} = m^{(j)}+\log(l^{(j)})$。

In the simple case of 2 blocks in Section 2.3, the online softmax trick now becomes:

$$
\begin{aligned}
m^{(1)} &= \text{rowmax}(\mathbf{S}^{(1)}) \in \mathbb{R}^{B_r} \\
\ell^{(1)} &= \text{rowsum}(e^{\mathbf{S}^{(1)} - m^{(1)}}) \in \mathbb{R}^{B_r} \\
\tilde{\mathbf{O}^{(1)}} &= e^{\mathbf{S}^{(1)} - m^{(1)}} \mathbf{V}^{(1)} \in \mathbb{R}^{B_r \times d} \\
m^{(2)} &= \max(m^{(1)}, \text{rowmax}(\mathbf{S}^{(2)})) = m \\
\ell^{(2)} &= e^{m^{(1)} - m^{(2)}} \ell^{(1)} + \text{rowsum}(e^{\mathbf{S}^{(2)} - m^{(2)}}) = \text{rowsum}(e^{\mathbf{S}^{(1)} - m}) + \text{rowsum}(e^{\mathbf{S}^{(2)} - m}) = \ell \\
\tilde{\mathbf{P}}^{(2)} &= \text{diag}(\ell^{(2)})^{-1} e^{\mathbf{S}^{(2)} - m^{(2)}} \\
\tilde{\mathbf{O}}^{(2)} &= \text{diag}(e^{m^{(1)} - m^{(2)}})^{-1} \tilde{\mathbf{O}}^{(1)} + e^{\mathbf{S}^{(2)} - m^{(2)}} \mathbf{V}^{(2)} = e^{s^{(1)} - m} \mathbf{V}^{(1)} + e^{s^{(2)} - m} \mathbf{V}^{(2)} \\
\mathbf{O}^{(2)} &= \text{diag}(\ell^{(2)})^{-1} \tilde{\mathbf{O}}^{(2)} = \mathbf{O}.
\end{aligned}
$$

We describe the full FlashAttention-2 forward pass in Algorithm 1.

Algorithm 1 FlashAttention-2 forward pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, block sizes B_c, B_r.

1: Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
2: Divide the output $\mathbf{O} \in \mathbb{R}^{N \times d}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, and divide the logsumexp L into T_r blocks $L_i, \dots, L_{T_r}$ of size B_r each.
3: **for** $1 \le i \le T_r$ **do**
4: Load $\mathbf{Q}_i$ from HBM to on-chip SRAM.
5: On chip, initialize $\mathbf{O}_i^{(0)} = (0)_{B_r \times d} \in \mathbb{R}^{B_r \times d}, \ell_i^{(0)} = (0)_{B_r} \in \mathbb{R}^{B_r}, m_i^{(0)} = (-\infty)_{B_r} \in \mathbb{R}^{B_r}$.
6: **for** $1 \le j \le T_c$ **do**
7: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
8: On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
9: On chip, compute $m_i^{(j)} = \max(m_i^{(j-1)}, \text{rowmax}(\mathbf{S}_i^{(j)})) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - m_i^{(j)}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\ell_i^{(j)} = e^{m_i^{j-1} - m_i^{(j)}} \ell_i^{(j-1)} + \text{rowsum}(\tilde{\mathbf{P}}_i^{(j)}) \in \mathbb{R}^{B_r}$.
10: On chip, compute $\mathbf{O}_i^{(j)} = \text{diag}(e^{m_i^{(j-1)} - m_i^{(j)}})^{-1} \mathbf{O}_i^{(j-1)} + \tilde{\mathbf{P}}_i^{(j)} \mathbf{V}_j$.
11: **end for**
12: On chip, compute $\mathbf{O}_i = \text{diag}(\ell_i^{(T_c)})^{-1} \mathbf{O}_i^{(T_c)}$.
13: On chip, compute $L_i = m_i^{(T_c)} + \log(\ell_i^{(T_c)})$.
14: Write $\mathbf{O}_i$ to HBM as the i-th block of $\mathbf{O}$.
15: Write L_i to HBM as the i-th block of L.
16: **end for**
17: Return the output $\mathbf{O}$ and the logsumexp L.

Causal masking. One common use case of attention is in auto-regressive language modeling, where we need to apply a causal mask to the attention matrix $\mathbf{S}$ (i.e., any entry $\mathbf{S}_{ij}$ with $j > i$ is set to $-\infty$).

1. As FlashAttention and FlashAttention-2 already operate by blocks, for any blocks where all the column indices are more than the row indices (approximately half of the blocks for large sequence length), we can skip the computation of that block. This leads to around 1.7-1.8× speedup compared to attention without the causal mask.

2. We do not need to apply the causal mask for blocks whose row indices are guaranteed to be strictly less than the column indices. This means that for each row, we only need apply causal mask to 1 block (assuming square block).

在2.3节中仅包含2个块的简单场景下，在线softmax技巧的具体实现方式如下：

$$
\begin{aligned}
m^{(1)} &= \text{rowmax}(\mathbf{S}^{(1)}) \in \mathbb{R}^{B_r} \\
\ell^{(1)} &= \text{rowsum}(e^{\mathbf{S}^{(1)} - m^{(1)}}) \in \mathbb{R}^{B_r} \\
\tilde{\mathbf{O}^{(1)}} &= e^{\mathbf{S}^{(1)} - m^{(1)}} \mathbf{V}^{(1)} \in \mathbb{R}^{B_r \times d} \\
m^{(2)} &= \max(m^{(1)}, \text{rowmax}(\mathbf{S}^{(2)})) = m \\
\ell^{(2)} &= e^{m^{(1)} - m^{(2)}} \ell^{(1)} + \text{rowsum}(e^{\mathbf{S}^{(2)} - m^{(2)}}) = \text{rowsum}(e^{\mathbf{S}^{(1)} - m}) + \text{rowsum}(e^{\mathbf{S}^{(2)} - m}) = \ell \\
\tilde{\mathbf{P}}^{(2)} &= \text{diag}(\ell^{(2)})^{-1} e^{\mathbf{S}^{(2)} - m^{(2)}} \\
\tilde{\mathbf{O}}^{(2)} &= \text{diag}(e^{m^{(1)} - m^{(2)}})^{-1} \tilde{\mathbf{O}}^{(1)} + e^{\mathbf{S}^{(2)} - m^{(2)}} \mathbf{V}^{(2)} = e^{s^{(1)} - m} \mathbf{V}^{(1)} + e^{s^{(2)} - m} \mathbf{V}^{(2)} \\
\mathbf{O}^{(2)} &= \text{diag}(\ell^{(2)})^{-1} \tilde{\mathbf{O}}^{(2)} = \mathbf{O}.
\end{aligned}
$$

我们在算法1中详细描述了完整的FlashAttention-2前向传播过程。

算法1 FlashAttention-2版本的前向传播

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, block sizes B_c, B_r.

1: Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
2: Divide the output $\mathbf{O} \in \mathbb{R}^{N \times d}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, and divide the logsumexp L into T_r blocks $L_i, \dots, L_{T_r}$ of size B_r each.
3: **for** $1 \le i \le T_r$ **do**
4: Load $\mathbf{Q}_i$ from HBM to on-chip SRAM.
5: On chip, initialize $\mathbf{O}_i^{(0)} = (0)_{B_r \times d} \in \mathbb{R}^{B_r \times d}, \ell_i^{(0)} = (0)_{B_r} \in \mathbb{R}^{B_r}, m_i^{(0)} = (-\infty)_{B_r} \in \mathbb{R}^{B_r}$.
6: **for** $1 \le j \le T_c$ **do**
7: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
8: On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
9: On chip, compute $m_i^{(j)} = \max(m_i^{(j-1)}, \text{rowmax}(\mathbf{S}_i^{(j)})) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - m_i^{(j)}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\ell_i^{(j)} = e^{m_i^{j-1} - m_i^{(j)}} \ell_i^{(j-1)} + \text{rowsum}(\tilde{\mathbf{P}}_i^{(j)}) \in \mathbb{R}^{B_r}$.
10: On chip, compute $\mathbf{O}_i^{(j)} = \text{diag}(e^{m_i^{(j-1)} - m_i^{(j)}})^{-1} \mathbf{O}_i^{(j-1)} + \tilde{\mathbf{P}}_i^{(j)} \mathbf{V}_j$.
11: **end for**
12: On chip, compute $\mathbf{O}_i = \text{diag}(\ell_i^{(T_c)})^{-1} \mathbf{O}_i^{(T_c)}$.
13: On chip, compute $L_i = m_i^{(T_c)} + \log(\ell_i^{(T_c)})$.
14: Write $\mathbf{O}_i$ to HBM as the i-th block of $\mathbf{O}$.
15: Write L_i to HBM as the i-th block of L.
16: **end for**
17: Return the output $\mathbf{O}$ and the logsumexp L.

因果掩码机制。注意力机制的一个常见应用场景是自回归语言建模，在这类任务中我们需要对注意力矩阵应用因果掩码机制$\mathbf{S}$ （即所有满足 $j > i$ 为 $-\infty$的$\mathbf{S}$元素都会被设置）。

1. 由于FlashAttention和FlashAttention-2版本本身就是按块处理的，对于那些所有列索引都大于行索引的块（在序列长度较大的情况下约占半数），我们可以直接跳过对该块的计算。这样一来，与未使用因果掩码机制的注意力机制相比，其加速比可达约1.7到1.8× 倍。

2. 对于那些行索引必然严格小于列索引的块，我们无需应用因果掩码。也就是说，在每行中，只需对1个块应用因果掩码即可（假设为正方形块）。

Correctness, runtime, and memory requirement. As with FlashAttention, Algorithm 1 returns the correct output $\mathbf{O} = \text{softmax}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V}$ (with no approximation), using $O(N^2 d)$ FLOPs and requires $O(N)$ additional memory beyond inputs and output (to store the logsumexp L). The proof is almost the same as the proof of Dao et al. [5, Theorem 1], so we omit it here.

3.1.2 Backward pass

The backward pass of FlashAttention-2 is almost the same as that of FlashAttention. We make a minor tweak to only use the row-wise logsumexp L instead of both the row-wise max and row-wise sum of exponentials in the softmax. We include the backward pass description in Algorithm 2 for completeness.

Algorithm 2 FlashAttention-2 Backward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ in HBM, vector $L \in \mathbb{R}^N$ in HBM, block sizes B_c, B_r.

Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide $\mathbf{dO}$ into T_r blocks $\mathbf{dO}_i, \dots, \mathbf{dO}_{T_r}$ of size $B_r \times d$ each, and divide L into T_r blocks $L_i, \dots, L_{T_r}$ of size B_r each.
Initialize $\mathbf{dQ} = (0)_{N \times d}$ in HBM and divide it into T_r blocks $\mathbf{dQ}_1, \dots, \mathbf{dQ}_{T_r}$ of size $B_r \times d$ each. Divide $\mathbf{dK}, \mathbf{dV} \in \mathbb{R}^{N \times d}$ in to T_c blocks $\mathbf{dK}_1, \dots, \mathbf{dK}_{T_c}$ and $\mathbf{dV}_1, \dots, \mathbf{dV}_{T_c}$, of size $B_c \times d$ each.
Compute $D = \text{rowsum}(\mathbf{dO} \circ \mathbf{O}) \in \mathbb{R}^d$ (pointwise multiply), write D to HBM and divide it into T_r blocks $D_1, \dots, D_{T_r}$ of size B_r each.
for $1 \le j \le T_c$ **do**
  Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
  Initialize $\mathbf{dK}_j = (0)_{B_c \times d}, \mathbf{dV}_j = (0)_{B_c \times d}$ on SRAM.
  for $1 \le i \le T_r$ **do**
    Load $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{dO}_i, \mathbf{dQ}_i, L_i, D_i$ from HBM to on-chip SRAM.
    On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $\mathbf{P}_i^{(j)} = \exp(\mathbf{S}_{ij} - L_i) \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $\mathbf{dV}_j \leftarrow \mathbf{dV}_j + (\mathbf{P}_i^{(j)})^\top \mathbf{dO}_i \in \mathbb{R}^{B_c \times d}$.
    On chip, compute $\mathbf{dP}_i^{(j)} = \mathbf{dO}_i \mathbf{V}_j^\top \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $\mathbf{dS}_i^{(j)} = \mathbf{P}_i^{(j)} \circ (\mathbf{dP}_i^{(j)} - D_i) \in \mathbb{R}^{B_r \times B_c}$.
    Load $\mathbf{dQ}_i$ from HBM to SRAM, then on chip, update $\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \mathbf{dS}_i^{(j)} \mathbf{K}_j \in \mathbb{R}^{B_r \times d}$, and write back to HBM.
    On chip, compute $\mathbf{dK}_j \leftarrow \mathbf{dK}_j + \mathbf{dS}_i^{(j)\top} \mathbf{Q}_i \in \mathbb{R}^{B_c \times d}$.
  end for
  Write $\mathbf{dK}_j, \mathbf{dV}_j$ to HBM.
end for
Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.

Multi-query attention and grouped-query attention. Multi-query attention (MQA) [15] and grouped-query attention (GQA) [1] are variants of attention where multiple heads of query attend to the same head of key and value, in order to reduce the size of KV cache during inference. Instead of having to duplicate the key and value heads for the computation, we implicitly manipulate the indices into the head to perform the same computation. In the backward pass, we need to sum the gradients $\mathbf{dK}$ and $\mathbf{dV}$ across different heads that were implicitly duplicated.

3.2 Parallelism

The first version of FlashAttention parallelizes over batch size and number of heads. We use 1 thread block to process one attention head, and there are overall batch size · number of heads thread blocks. Each thread block is scheduled to run on a streaming multiprocessor (SM), and there are 108 of these SMs on

正确性、运行时间以及内存需求。与FlashAttention算法类似，算法1能够输出正确的结果：$\mathbf{O} = \text{softmax}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V}$（无需任何近似处理），该算法仅需 $O(N^2 d)$次浮点运算次数，除了输入和输出数据外还需 $O(N)$额外的内存用于存储对数和指数运算的结果 L。其证明方法与Dao等人 [5, 定理 1],的证明几乎相同，因此此处不再赘述。

3.1.2 后向传播

FlashAttention-2的向后传播过程与FlashAttention几乎相同。我们仅做了一处细微调整，即在softmax计算中仅使用按行进行的对数和指数运算 L ，而非同时使用按行求得的指数最大值和按行求得的指数和。为保证完整性，我们将在算法2中列出该向后传播过程的详细描述。

Algorithm 2 FlashAttention-2 Backward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ in HBM, vector $L \in \mathbb{R}^N$ in HBM, block sizes B_c, B_r.

Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide $\mathbf{dO}$ into T_r blocks $\mathbf{dO}_i, \dots, \mathbf{dO}_{T_r}$ of size $B_r \times d$ each, and divide L into T_r blocks $L_i, \dots, L_{T_r}$ of size B_r each.
Initialize $\mathbf{dQ} = (0)_{N \times d}$ in HBM and divide it into T_r blocks $\mathbf{dQ}_1, \dots, \mathbf{dQ}_{T_r}$ of size $B_r \times d$ each. Divide $\mathbf{dK}, \mathbf{dV} \in \mathbb{R}^{N \times d}$ in to T_c blocks $\mathbf{dK}_1, \dots, \mathbf{dK}_{T_c}$ and $\mathbf{dV}_1, \dots, \mathbf{dV}_{T_c}$, of size $B_c \times d$ each.
Compute $D = \text{rowsum}(\mathbf{dO} \circ \mathbf{O}) \in \mathbb{R}^d$ (pointwise multiply), write D to HBM and divide it into T_r blocks $D_1, \dots, D_{T_r}$ of size B_r each.
for $1 \le j \le T_c$ **do**
  Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
  Initialize $\mathbf{dK}_j = (0)_{B_c \times d}, \mathbf{dV}_j = (0)_{B_c \times d}$ on SRAM.
  for $1 \le i \le T_r$ **do**
    Load $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{dO}_i, \mathbf{dQ}_i, L_i, D_i$ from HBM to on-chip SRAM.
    On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $\mathbf{P}_i^{(j)} = \exp(\mathbf{S}_{ij} - L_i) \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $\mathbf{dV}_j \leftarrow \mathbf{dV}_j + (\mathbf{P}_i^{(j)})^\top \mathbf{dO}_i \in \mathbb{R}^{B_c \times d}$.
    On chip, compute $\mathbf{dP}_i^{(j)} = \mathbf{dO}_i \mathbf{V}_j^\top \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $\mathbf{dS}_i^{(j)} = \mathbf{P}_i^{(j)} \circ (\mathbf{dP}_i^{(j)} - D_i) \in \mathbb{R}^{B_r \times B_c}$.
    Load $\mathbf{dQ}_i$ from HBM to SRAM, then on chip, update $\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \mathbf{dS}_i^{(j)} \mathbf{K}_j \in \mathbb{R}^{B_r \times d}$, and write back to HBM.
    On chip, compute $\mathbf{dK}_j \leftarrow \mathbf{dK}_j + \mathbf{dS}_i^{(j)\top} \mathbf{Q}_i \in \mathbb{R}^{B_c \times d}$.
  end for
  Write $\mathbf{dK}_j, \mathbf{dV}_j$ to HBM.
end for
Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.

多查询注意力机制与分组查询注意力机制。多查询注意力机制（MQA） [15] 和分组查询注意力机制（GQA） [1] 都属于注意力机制的变体，其特点是让多个查询头同时作用于同一个键值头，以此在推理过程中缩小KV缓存的大小。由于无需为计算重复生成键值头，我们可以通过对头部索引进行间接操作来完成相同的计算任务。在后续的后向传播阶段，则需要将对这些被间接复制的不同头部对应的梯度$\mathbf{dK}$ 与$\mathbf{dV}$ 进行求和。

3.2 并行性

FlashAttention的第一个版本通过对批次大小和注意力头数量进行并行处理来实现优化。我们使用1个线程块来处理一个注意力头，因此总的线程块数量为批次大小乘以注意力头数量。每个线程块都会被调度在流式多处理器上运行，而某款A100显卡上就配备了108个这样的流式多处理器。

an A100 GPU for example. This scheduling is efficient when this number is large (say ≥ 80), since we can effectively use almost all of the compute resources on the GPU.

In the case of long sequences (which usually means small batch sizes or small number of heads), to make better use of the multiprocessors on the GPU, we now additionally parallelize over the sequence length dimension. This results in significant speedup for this regime.

Forward pass. We see that the outer loop (over sequence length) is embarrassingly parallel, and we schedule them on different thread blocks that do not need to communicate with each other. We also parallelize over the batch dimension and number of heads dimension, as done in FlashAttention. The increased parallelism over sequence length helps improve occupancy (fraction of GPU resources being used) when the batch size and number of heads are small, leading to speedup in this case.

These ideas of swapping the order of the loop (outer loop over row blocks and inner loop over column blocks, instead of the other way round in the original FlashAttention paper), as well as parallelizing over the sequence length dimension were first suggested and implemented by Phil Tillet in the Triton [17] implementation.[3]

Backward pass. Notice that the only shared computation between different column blocks is in update $\mathbf{dQ}$ in Algorithm 2, where we need to load $\mathbf{dQ}_i$ from HBM to SRAM, then on chip, update $\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \mathbf{dS}_i^{(j)}\mathbf{K}_j$, and write back to HBM. We thus parallelize over the sequence length dimension as well, and schedule 1 thread block for each column block of the backward pass. We use atomic adds to communicate between different thread blocks to update $\mathbf{dQ}$.

We describe the parallelization scheme in Fig. 2.

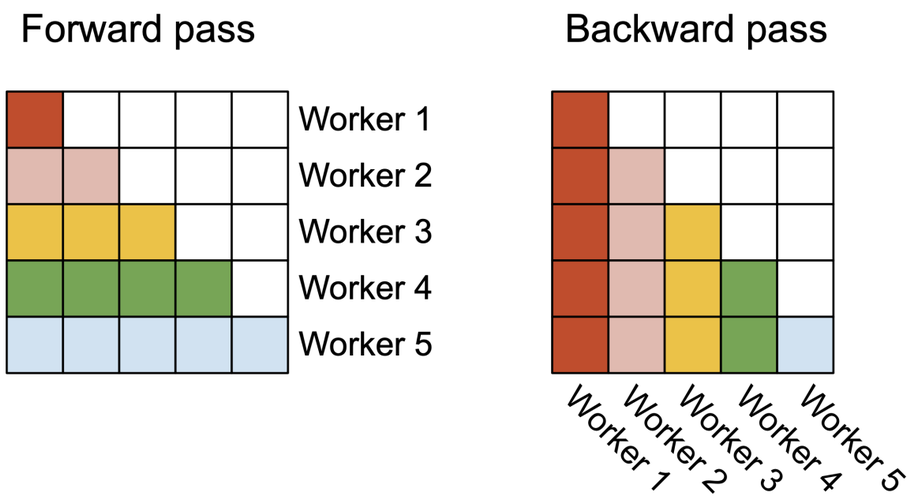


Figure 2: In the forward pass (left), we parallelize the workers (thread blocks) where each worker takes care of a block of rows of the attention matrix. In the backward pass (right), each worker takes care of a block of columns of the attention matrix.

[3] https://github.com/openai/triton/blob/main/python/tutorials/06-fused-attention.py

例如在某款A100显卡上就是如此。当线程块的数量较大（比如 ≥ 80）时，这种调度方式效率极高，因为我们可以几乎充分利用该显卡上的所有计算资源。

对于长序列场景（通常意味着批量大小较小或注意力头数量较少），为了更充分地利用图形处理器上的多处理器资源，我们现在还进一步在序列长度维度上实现了并行处理。这样一来，这类场景下的加速比就有了显著提升。

前向传播。我们可以看到，针对序列长度的外层循环具备极高的并行性，因此我们会将其分配到无需相互通信的不同线程块中执行。与FlashAttention中的做法一致，我们同样对批次维度及注意力头数量维度进行并行化处理。当批次大小和注意力头数量较小时，序列长度层面的更高并行性有助于提升GPU资源利用率，从而实现加速效果。

在Triton [17]实现版本中，Phil Tillet首次提出了这些思路——即调整循环顺序（用外层循环处理行块、内层循环处理列块，而非原始FlashAttention论文中的相反顺序），并在序列长度维度上实现并行处理，随后也将这些方案付诸实践。[3]

进行后向传播。请注意，不同列块之间唯一的共享计算发生在算法2中的$\mathbf{dQ}$更新环节，此时我们需要将$\mathbf{dQ}_i$从高带宽内存加载到片上静态随机存取存储器中，然后在芯片内部更新$\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \mathbf{dS}^{(j)}{}_i\mathbf{K}_j$，最后再将结果写回高带宽内存。因此，我们也在序列长度维度上实现并行处理，为后向传播的每个列块分配1个线程块。为了在各个线程块之间协同更新$\mathbf{dQ}$，我们采用了原子加法操作来进行数据传递。

我们在图2中介绍了该并行化方案。

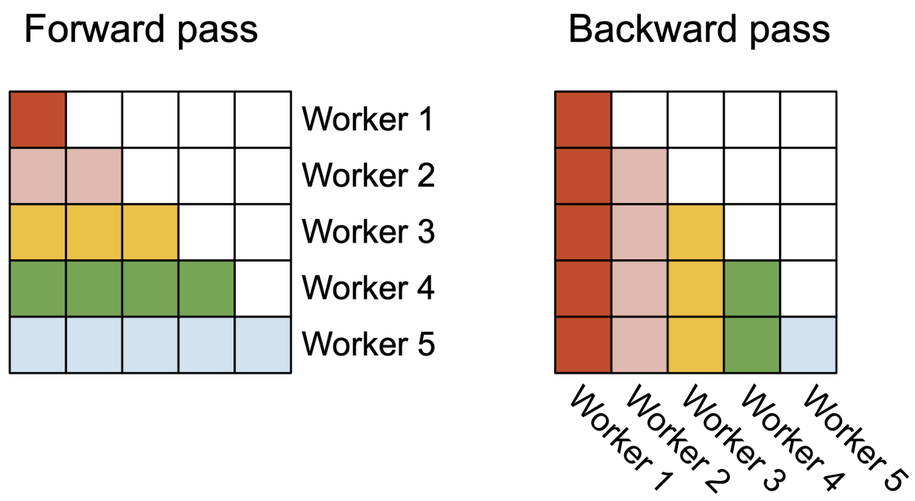


图2：在前向传播阶段（左侧），我们会对线程块进行并行化处理，每个线程块负责处理注意力矩阵中的一组行数据。而在后向传播阶段（右侧），每个线程块则负责处理注意力矩阵中的一组列数据。

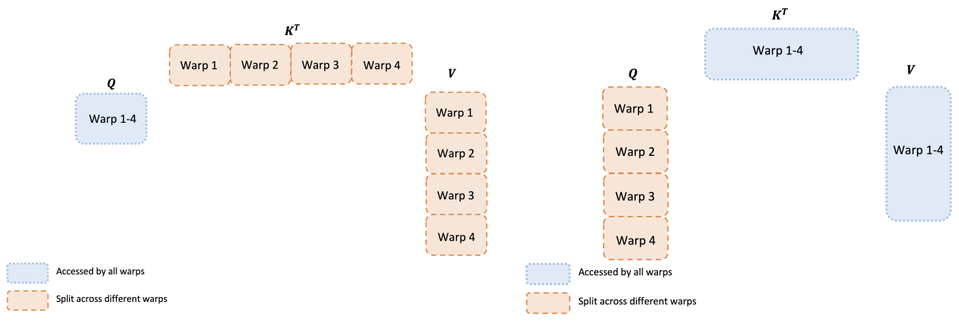
[3] https://github.com/openai/triton/blob/main/python/tutorials/06-fused-attention.py

3.3 Work Partitioning Between Warps

As Section 3.2 describe how we schedule thread blocks, even within each thread block, we also have to decide how to partition the work between different warps. We typically use 4 or 8 warps per thread block, and the partitioning is described in Fig. 3.

Forward pass. For each block, FlashAttention splits $\mathbf{K}$ and $\mathbf{V}$ across 4 warps while keeping $\mathbf{Q}$ accessible by all warps. Each warp multiplies to get a slice of $\mathbf{QK}^\top$, then they need to multiply with a slice of $\mathbf{V}$ and communicate to add up the result. This is referred to as the "split-K" scheme. However, this is inefficient since all warps need to write their intermediate results out to shared memory, synchronize, then add up the intermediate results. These shared memory reads/writes slow down the forward pass in FlashAttention.

In FlashAttention-2, we instead split $\mathbf{Q}$ across 4 warps while keeping $\mathbf{K}$ and $\mathbf{V}$ accessible by all warps. After each warp performs matrix multiply to get a slice of $\mathbf{QK}^\top$, they just need to multiply with their shared slice of $\mathbf{V}$ to get their corresponding slice of the output. There is no need for communication between warps. The reduction in shared memory reads/writes yields speedup (Section 4).



(a) FlashAttention (b) FlashAttention-2

Figure 3: Work partitioning between different warps in the forward pass

Backward pass. Similarly for the backward pass, we choose to partition the warps to avoid the "split-K" scheme. However, it still requires some synchronization due to the more complicated dependency between all the different inputs and gradients $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO}, \mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$. Nevertheless, avoiding "split-K" reduces shared memory reads/writes and again yields speedup (Section 4).

Tuning block sizes Increasing block sizes generally reduces shared memory loads/stores, but increases the number of registers required and the total amount of shared memory. Past a certain block size, register spilling causes significant slowdown, or the amount of shared memory required is larger than what the GPU has available, and the kernel cannot run at all. Typically we choose blocks of size $\{64, 128\} \times \{64, 128\}$, depending on the head dimension d and the device shared memory size.

We manually tune for each head dimensions since there are essentially only 4 choices for block sizes, but this could benefit from auto-tuning to avoid this manual labor. We leave this to future work.

4 Empirical Validation

We evaluate the impact of using FlashAttention-2 to train Transformer models.

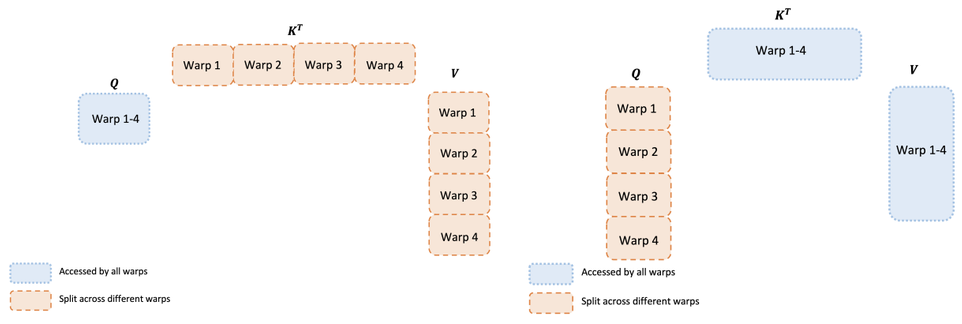
- **Benchmarking attention.** We measure the runtime of FlashAttention-2 across different sequence lengths and compare it to a standard implementation in PyTorch, FlashAttention, and FlashAttention in Triton. We confirm that FlashAttention-2 is 1.7-3.0× faster than FlashAttention, 1.3-2.5× faster than FlashAttention in Triton, and 3-10× faster than a standard attention implementation.

3.3 线程组之间的任务划分

正如3.2节所阐述的，我们在安排线程块的执行顺序时，即便在同一个线程块内部，也必须决定如何在不同线程组之间分配任务。通常每个线程块会包含4个或8个线程组，而具体的分配方式则如图3所示。

进行前向传播。对于每个块，FlashAttention算法会将$\mathbf{K}$ 和$\mathbf{V}$ 分配到4个线程组中，同时确保所有线程组都能访问$\mathbf{Q}$ 。每个线程组会相乘以得到$\mathbf{QK}^\top$的一个切片，之后它们需要再与$\mathbf{V}$ 的一个切片相乘，并通过通信将结果相加。这种方法被称为“拆分K”方案。然而，这种方式的效率较低，因为所有线程组都需要将自己的中间结果写入共享内存，进行同步处理，然后再将这些中间结果相加。这些频繁的共享内存读写操作会拖慢FlashAttention中的前向传播速度。

在FlashAttention-2中，我们改为将$\mathbf{Q}$ 数据分配到4个线程组中，同时确保所有线程组都能访问$\mathbf{K}$ 和$\mathbf{V}$ 数据。每个线程组完成矩阵乘法运算得到$\mathbf{QK}^\top$的一个切片后，只需再与它们共享的$\mathbf{V}$ 切片相乘，即可得到对应的输出切片。这样一来，各个线程组之间无需进行任何通信。由于共享内存的读写操作大幅减少，从而实现了加速比的提升（见第4节）。



(a) FlashAttention算法 (b) FlashAttention-2版本

图3：前向传播过程中不同线程组之间的任务划分

后向传播。在后向传播过程中，我们同样选择对线程组进行划分，以避免采用“拆分K”的方案。不过，由于各种不同的输入与梯度之间存在更为复杂的依赖关系，$\mathbf{Q}$、$\mathbf{K}$、$\mathbf{V}$、$\mathbf{O}$、$\mathbf{dO}$、$\mathbf{dQ}$、$\mathbf{dK}$、$\mathbf{dV}$，这一过程仍需要一定的同步操作。尽管如此，避开“拆分K”方案能够减少共享内存的读写操作，从而再次实现加速比的提升（见第4节）。

块大小的调整 通常情况下，增大块大小可以减少对共享内存的读写操作，但会增加所需的寄存器数量以及共享内存的总容量。一旦超过某个特定块大小，寄存器溢出问题就会导致性能显著下降；或者所需的共享内存容量会超出图形处理器可用的容量，进而使得内核根本无法运行。一般来说，我们会根据头维度 d 和设备共享内存容量来选择大小为$\{64，128\} \times \{64，128\}$的块。

由于块大小基本上只有4种可选方案，我们只能针对每个头维度进行手动调优；不过借助自动调优技术或许可以避免这种繁琐的手工操作，这方面的工作我们将留待后续研究。

4 实证验证

我们研究了在训练Transformer模型时采用FlashAttention-2算法所带来的影响。

- **我们正在对注意力机制进行性能测试**。我们测量了不同序列长度下FlashAttention-2的运行时间，并将其与PyTorch中的标准实现、FlashAttention本身以及Triton中的FlashAttention实现进行了对比。测试结果表明，FlashAttention-2的运行速度比FlashAttention快1.7到3.0× 倍，比Triton中的FlashAttention快1.3到2.5×倍，更比标准的注意力机制实现快3-10× 倍。

FlashAttention-2 reaches up to 230 TFLOPs/s, 73% of the theoretical maximum TFLOPs/s on A100 GPUs.

- **End-to-end training speed** When used end-to-end to train GPT-style models of size 1.3B and 2.7B on sequence lengths either 2k or 8k, FlashAttention-2 yields up to 1.3× speedup compared to FlashAttention and 2.8× speedup compared to a baseline without FlashAttention. FlashAttention-2 reaches up to 225 TFLOPs/s (72% model FLOPs utilization) per A100 GPU.

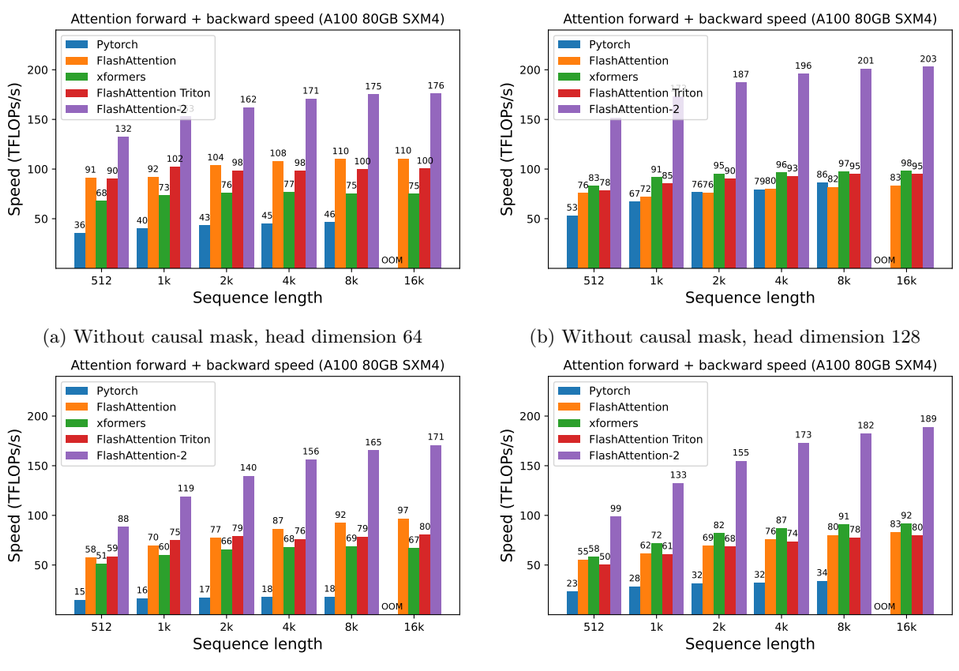
4.1 Benchmarking Attention

We measure the runtime of different attention methods on an A100 80GB SXM4 GPU for different settings (without / with causal mask, head dimension 64 or 128). We report the results in Fig. 4, Fig. 5 and Fig. 6, showing that FlashAttention-2 is around 2× faster than FlashAttention and FlashAttention in `xformers` (the "cutlass" implementation). FlashAttention-2 is around 1.3-1.5× faster than FlashAttention in Triton in the forward pass and around 2× faster in the backward pass. Compared to a standard attention implementation in PyTorch, FlashAttention-2 can be up to 10× faster.

Benchmark setting: we vary the sequence length from 512, 1k, ..., 16k, and set batch size so that the total number of tokens is 16k. We set hidden dimension to 2048, and head dimension to be either 64 or 128 (i.e., 32 heads or 16 heads). To calculate the FLOPs of the forward pass, we use:

$$4 \cdot \text{seqlen}^2 \cdot \text{head dimension} \cdot \text{number of heads}.$$

With causal mask, we divide this number by 2 to account for the fact that approximately only half of the entries are calculated. To get the FLOPs of the backward pass, we multiply the forward pass FLOPs by 2.5 (since there are 2 matmuls in the forward pass and 5 matmuls in the backward pass, due to recomputation).



(a) Without causal mask, head dimension 64

(b) Without causal mask, head dimension 128

(c) With causal mask, head dimension 64

(d) With causal mask, head dimension 128

Figure 4: Attention forward + backward speed on A100 GPU

FlashAttention-2的每秒浮点运算次数高达230 TFLOPs/s，这一数值相当于A100显卡上理论最大值的73%。

• **端到端训练速度**当用于对规模为13亿和27亿参数的GPT风格模型进行端到端训练，且序列长度分别为2千或8千时，FlashAttention-2相比FlashAttention可实现高达1.3× 倍的加速比，而相比未采用该算法的基准方案则可实现2.8× 倍的加速比。在每台A100显卡上，FlashAttention-2的浮点运算次数最高可达225每秒万亿次，模型浮点运算次数的利用率约为72%。

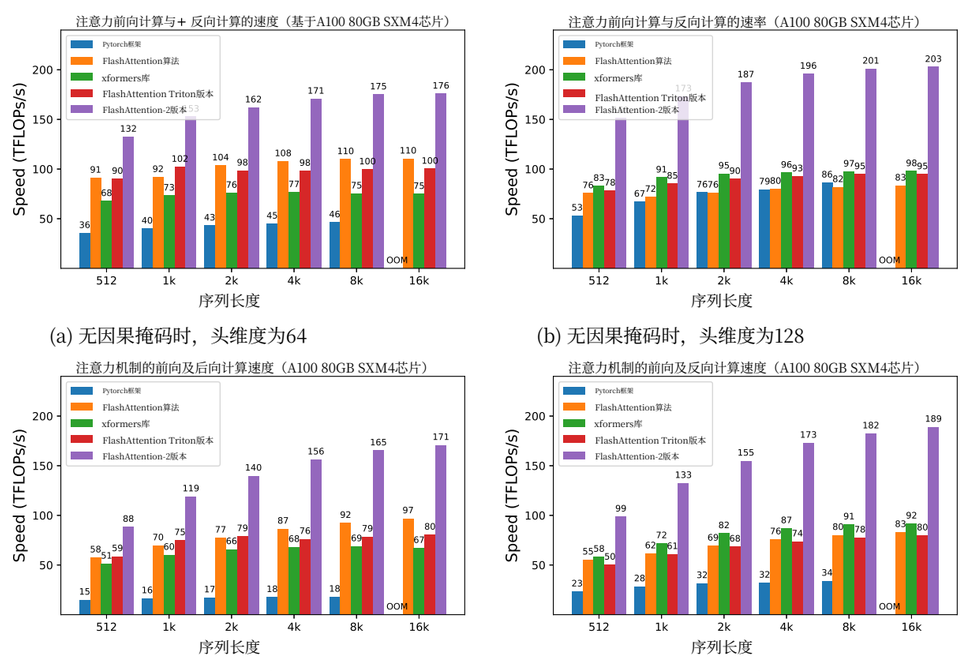
4.1 Benchmarking Attention

我们在A100 80GB SXM4图形处理器上，针对不同的配置（不使用/使用因果掩码、头维度为64或128）测试了各类注意力机制的运行时间。图4、图5和图6展示了测试结果，可以看出FlashAttention-2的运行速度比FlashAttention以及xformers中的FlashAttention实现（即“cutlass”版本）快约 2× 倍。在前向传播过程中，FlashAttention-2的速度比Triton中的FlashAttention快约1.3-1.5× 倍，而在后向传播过程中的速度则快约 2× 倍。与PyTorch中的标准注意力机制实现相比，FlashAttention-2的运行速度甚至可快达 10× 倍。

基准测试设置：我们让序列长度在512、1千、……16千之间变化，并调整批量大小以确保token总数为16千。隐藏层维度设为2048，头维度则设置为64或128（即32个头或16个头）。为了计算前向传播的浮点运算次数，我们采用以下公式：

$$4 \cdot \text{seqlen}^2 \cdot \text{head dimension} \cdot \text{number of heads}.$$

借助因果掩码，我们需要将这一数值除以2，因为实际上仅有大约一半的元素会被计算。而要得出后向传播的浮点运算次数，只需将前向传播的浮点运算次数乘以2.5——这是因为前向传播中有2次矩阵乘法，而后向传播由于需要重新计算，共有5次矩阵乘法。



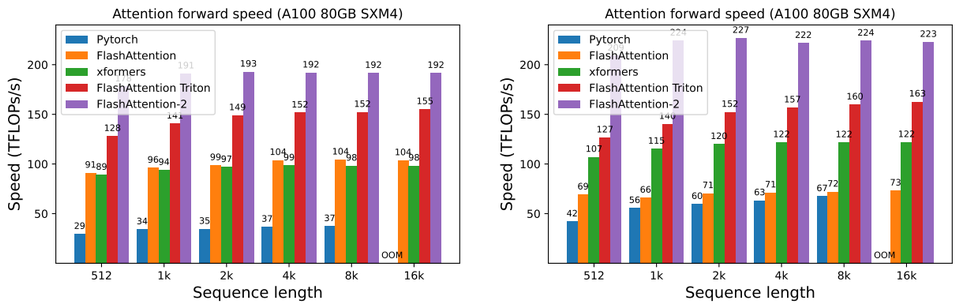
(a) 无因果掩码时，头维度为64

(b) 无因果掩码时，头维度为128

(c) 有因果掩码时，头维度为64

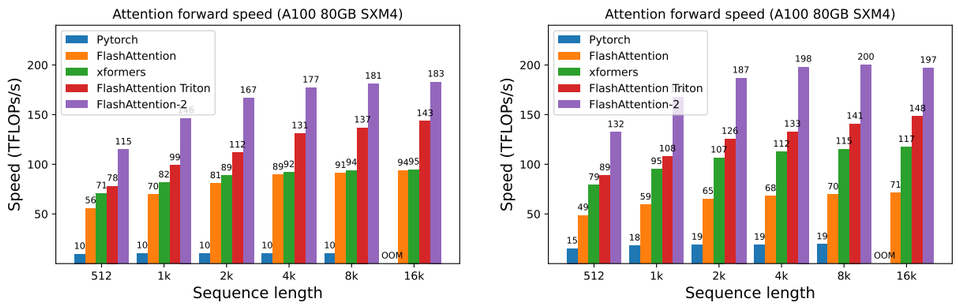
(d) 有因果掩码时，头维度为128

图4：在A100显卡上进行的注意力机制前向计算与 +反向计算的速度对比



(a) Without causal mask, head dimension 64

(b) Without causal mask, head dimension 128



(c) With causal mask, head dimension 64

(d) With causal mask, head dimension 128

Figure 5: Attention forward speed on A100 GPU

Just running the same implementation on H100 GPUs (using no special instructions to make use of new features such as TMA and 4th-gen Tensor Cores), we obtain up to 335 TFLOPs/s (Fig. 7). We expect that by using new instructions, we can obtain another 1.5x-2x speedup on H100 GPUs. We leave that to future work.

4.2 End-to-end Performance

We measure the training throughput of GPT-style models with either 1.3B or 2.7B parameters, on 8×A100 80GB SXM. As shown in Table 1, FlashAttention-2 yields 2.8× speedup compared to a baseline without FlashAttention and 1.3× speedup compared to FlashAttention-2, reaching up to 225 TFLOPs/s per A100 GPU.

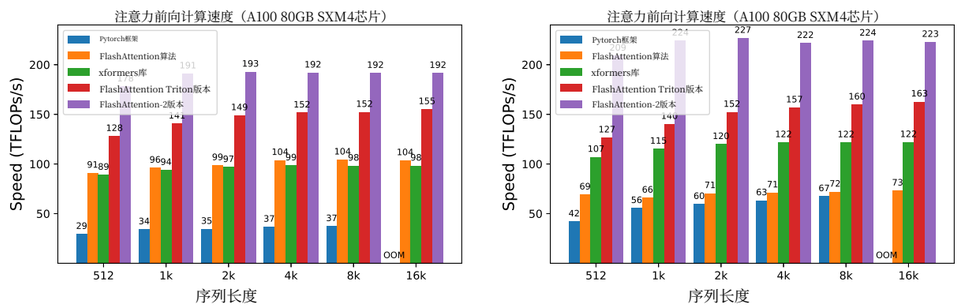
Note that we calculate the FLOPs by the formula, following Megatron-LM [16] (and many other papers and libraries):

$$6 \cdot \text{seqlen} \cdot \text{number of params} + 12 \cdot \text{number of layers} \cdot \text{hidden dim} \cdot \text{seqlen}^2.$$

The first term accounts for the FLOPs due to weight–input multiplication, and the second term accounts for the FLOPs due to attention. However, one can argue that the second term should be halved, as with causal mask we only need to compute approximately half the number of elements in attention. We choose to follow the formula from the literature (without dividing the attention FLOPs by 2) for consistency.

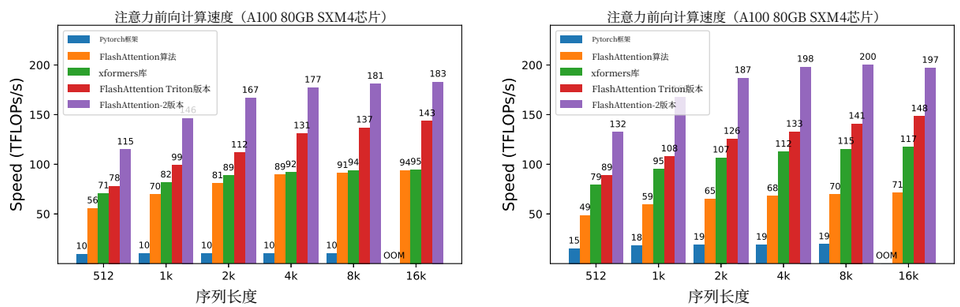
5 Discussion and Future Directions

FlashAttention-2 is 2× faster than FlashAttention, which means that we can train models with 16k longer context for the same price as previously training a 8k context model. We are excited about how this can



(a) 未应用因果掩码，头维度为64

(b) 未应用因果掩码，头维度为128



(c) 使用因果掩码，头维度为64。

(d) 使用因果掩码，头维度为128。

图5：在A100显卡上的注意力前向计算速度。

即便仅在H100显卡上运行相同的实现代码（且不使用任何特殊指令来利用TMA技术和第四代张量核心等新功能），其每秒浮点运算次数也能达到335万亿次/秒（见图7）。我们预计，通过运用新的指令集，还能在H100显卡上实现1.5倍到2倍的进一步加速，具体的优化工作将留待后续研究。

4.2 端到端性能测试

我们在 8×A100 80GB SXM硬件上，测试了参数量为13亿或27亿的GPT风格模型的训练吞吐量。如表1所示，相比未采用FlashAttention的基准方案，FlashAttention-2版本的加速比为2.8×；再与FlashAttention-2本身相比，其加速比为1.3×，每块A100显卡的浮点运算次数最高可达225每秒万亿次。

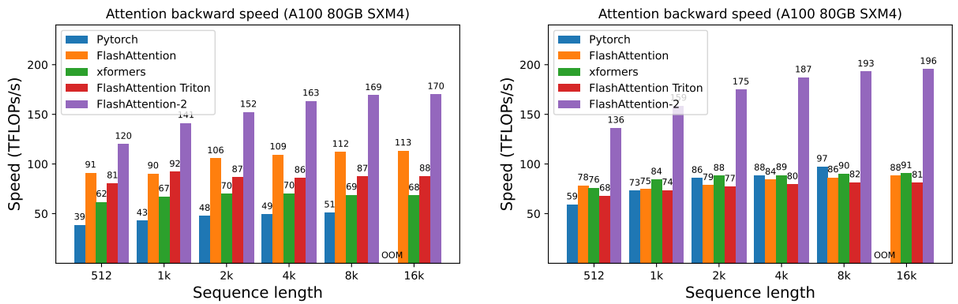
需注意的是，我们遵循Megatron-LM [16]（以及许多其他论文和库）中的公式来计算浮点运算次数。

$$6 \cdot \text{seqlen} \cdot \text{number of params} + 12 \cdot \text{number of layers} \cdot \text{hidden dim} \cdot \text{seqlen}^2.$$

第一项用于计算由权重与输入相乘所产生的浮点运算次数，而第二项则用于计算由注意力机制带来的浮点运算次数。不过也有观点认为第二项的数值应当减半，因为应用因果掩码时，实际上只需计算注意力机制中大约一半的元素对应的运算量即可。为保持一致性，我们选择沿用现有文献中的公式（不将注意力机制相关的浮点运算次数除以2）。

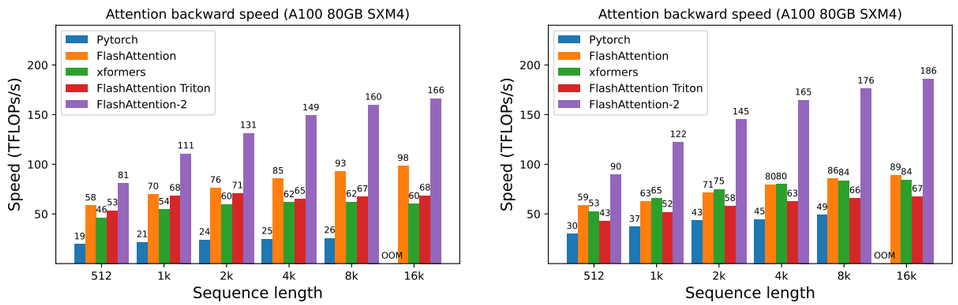
5 讨论与未来发展方向

FlashAttention-2版本的效率比FlashAttention算法高出 2× 倍，这意味着我们能够以与之前训练8千词上下文长度模型相同的成本，来训练具有16千词上下文长度的模型。我们对这一技术能带来的应用前景感到十分期待。



(a) Without causal mask, head dimension 64

(b) Without causal mask, head dimension 128



(c) With causal mask, head dimension 64

(d) With causal mask, head dimension 128

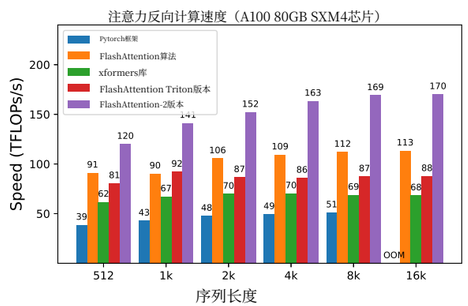
Figure 6: Attention backward speed on A100 GPU

Table 1: Training speed (TFLOPs/s/GPU) of GPT-style models on 8×A100 GPUs. FlashAttention-2 reaches up to 225 TFLOPs/s (72% model FLOPs utilization). We compare against a baseline running without FlashAttention.

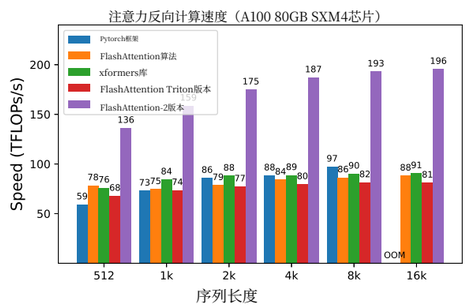
Model	Without FlashAttention	FlashAttention	FlashAttention-2
GPT3-1.3B 2k context	142 TFLOPs/s	189 TFLOPs/s	196 TFLOPs/s
GPT3-1.3B 8k context	72 TFLOPS/s	170 TFLOPs/s	220 TFLOPs/s
GPT3-2.7B 2k context	149 TFLOPs/s	189 TFLOPs/s	205 TFLOPs/s
GPT3-2.7B 8k context	80 TFLOPs/s	175 TFLOPs/s	225 TFLOPs/s

be used to understand long books and reports, high resolution images, audio and video. FlashAttention-2 will also speed up training, finetuning, and inference of existing models.

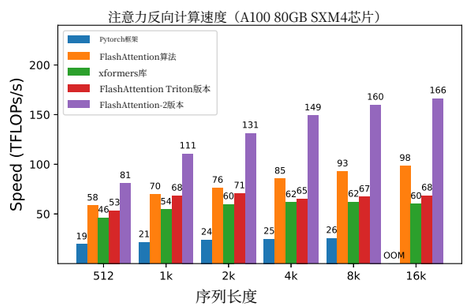
In the near future, we plan to collaborate with researchers and engineers to make FlashAttention widely applicable in different kinds of devices (e.g., H100 GPUs, AMD GPUs), as well as new data types such as FP8. As an immediate next step, we plan to optimize FlashAttention-2 for H100 GPUs to use new hardware features (TMA, 4th-gen Tensor Cores, fp8). Combining the low-level optimizations in FlashAttention-2 with high-level algorithmic changes (e.g., local, dilated, block-sparse attention) could allow us to train AI models with much longer context. We are also excited to work with compiler researchers to make these optimization techniques easily programmable.



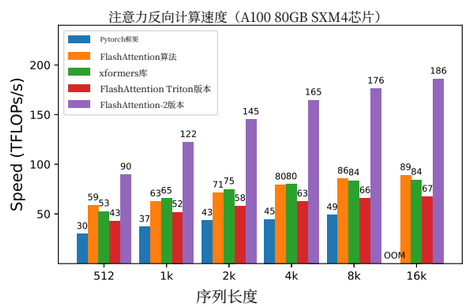
(a) 未应用因果掩码时，头维度为64



(b) 未应用因果掩码时，头维度为128



(c) 带有因果掩码，头维度为64。



(d) 带有因果掩码，头维度为128。

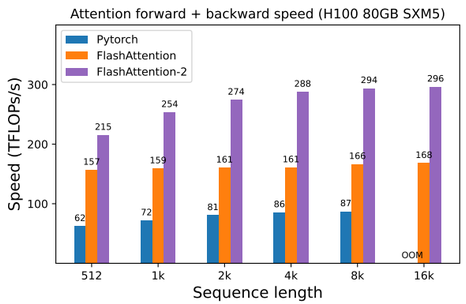
图6：在A100显卡上的注意力机制后向计算速度。

表1：在 8×台A100显卡上运行GPT风格模型的训练速度（每秒万亿次浮点运算次数/图形处理器）。FlashAttention-2版本的该速度可达到225每秒万亿次浮点运算次数，模型浮点运算次数的利用率可达72%。此处还将该版本与未采用FlashAttention算法的基准模型进行了对比。

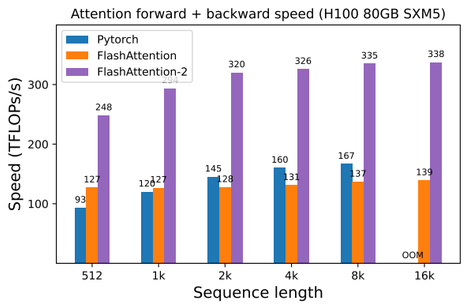
模型类型	未使用FlashAttention算法	FlashAttention算法	FlashAttention-2版本
GPT3-1.3B模型的2千上下文处理能力	142每秒万亿次浮点运算次数	189每秒万亿次浮点运算次数	196每秒万亿次浮点运算次数
GPT3-1.3B模型的8千词上下文长度	72每秒万亿次浮点运算次数	170每秒万亿次浮点运算次数	220每秒万亿次浮点运算次数
GPT3-2.7B版本的2千词上下文长度	149每秒万亿次浮点运算次数	189 每秒万亿次浮点运算次数	205 每秒万亿次浮点运算次数
GPT3-2.7B版本 8千词上下文长度	80 每秒万亿次浮点运算次数	175 每秒万亿次浮点运算次数	225 每秒万亿次浮点运算次数

它能够帮助人们更好地理解篇幅较长的书籍和报告，以及高分辨率图像、音频和视频文件。此外，FlashAttention-2还能加快现有模型的训练、微调及推理速度。

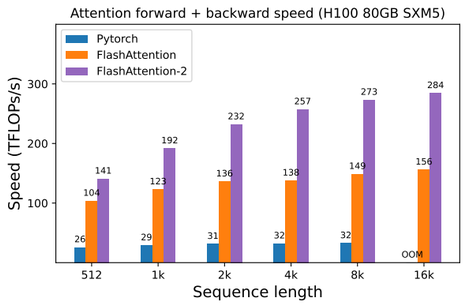
在不久的将来，我们计划与研究人员及工程师携手合作，推动FlashAttention在各类设备（如H100显卡、AMD GPU）以及FP8格式等新型数据类型上得到更广泛的应用。作为接下来的首要任务，我们打算针对H100显卡优化FlashAttention-2，使其能够充分利用新的硬件特性（如TMA技术、第四代张量核心、FP8格式）。通过将FlashAttention-2中的底层优化与高级算法改进（例如局部注意力、扩张注意力、分块稀疏注意力）相结合，我们有望训练出具备更长上下文长度的AI模型。同时，我们也期待能与编译器领域的专家合作，让这些优化技术更易于被程序化实现。



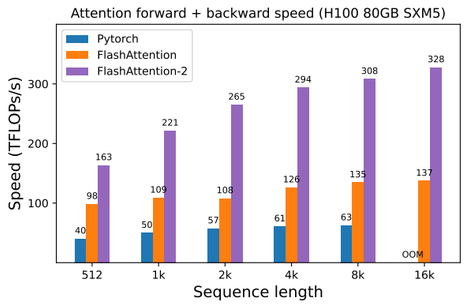
(a) Without causal mask, head dimension 64



(b) Without causal mask, head dimension 128



(c) With causal mask, head dimension 64



(d) With causal mask, head dimension 128

Figure 7: Attention forward + backward speed on H100 GPU

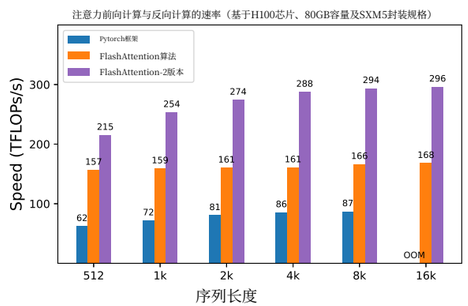
Acknowledgments

We thank Phil Tillet and Daniel Haziza, who have implemented versions of FlashAttention in Triton [17] and the `xformers` library [10]. FlashAttention-2 was motivated by exchange of ideas between different ways that attention could be implemented. We are grateful to the Nvidia CUTLASS team (especially Vijay Thakkar, Cris Cecka, Haicheng Wu, and Andrew Kerr) for their CUTLASS library, in particular the CUTLASS 3.x release, which provides clean abstractions and powerful building blocks for the implementation of FlashAttention-2. We thank Driss Guessous for integrating FlashAttention to PyTorch. FlashAttention-2 has benefited from helpful discussions with Phil Wang, Markus Rabe, James Bradbury, Young-Jun Ko, Julien Launay, Daniel Hesslow, Michaël Benesty, Horace He, Ashish Vaswani, and Erich Elsen. Thanks for Stanford CRFM and Stanford NLP for the compute support. We thank Dan Fu and Christopher Ré for their collaboration, constructive feedback, and constant encouragement on this line of work of designing hardware-efficient algorithms. We thank Albert Gu and Beidi Chen for their helpful suggestions on early drafts of this technical report.

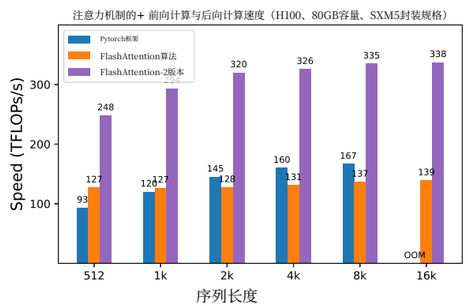
References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.

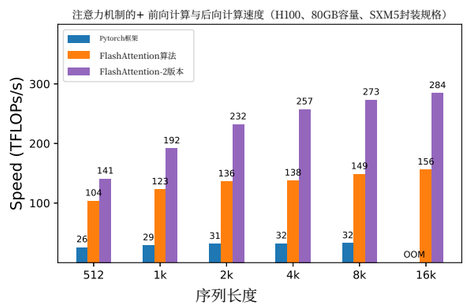
[2] Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.



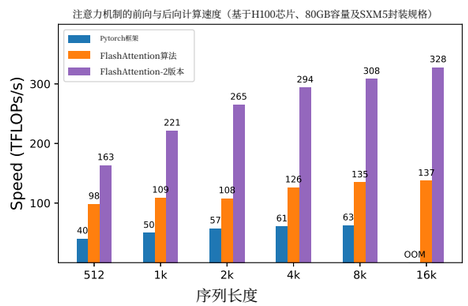
(a) 未应用因果掩码时，头维度为64



(b) 未应用因果掩码时，头维度为128



(c) 应用因果掩码时，头维度为64



(d) 应用因果掩码时，头维度为128

图7：H100显卡上的注意力前向计算 +与后向计算速度

致谢

We thank Phil Tillet and Daniel Haziza, who have implemented versions of FlashAttention in Triton [17] and the `xformers` library [10]. FlashAttention-2 was motivated by exchange of ideas between different ways that attention could be implemented. We are grateful to the Nvidia CUTLASS team (especially Vijay Thakkar, Cris Cecka, Haicheng Wu, and Andrew Kerr) for their CUTLASS library, in particular the CUTLASS 3.x release, which provides clean abstractions and powerful building blocks for the implementation of FlashAttention-2. We thank Driss Guessous for integrating FlashAttention to PyTorch. FlashAttention-2 has benefited from helpful discussions with Phil Wang, Markus Rabe, James Bradbury, Young-Jun Ko, Julien Launay, Daniel Hesslow, Michaël Benesty, Horace He, Ashish Vaswani, and Erich Elsen. Thanks for Stanford CRFM and Stanford NLP for the compute support. We thank Dan Fu and Christopher Ré for their collaboration, constructive feedback, and constant encouragement on this line of work of designing hardware-efficient algorithms. We thank Albert Gu and Beidi Chen for their helpful suggestions on early drafts of this technical report.

参考文献

[1] Joshua Ainslie、James Lee-Thorp、Michiel de Jong、Yury Zemlyanskiy、Federico Lebrón以及Sumit Sanghai。GQA：基于多头检查点训练广义多查询变换器模型。*arXiv*预印本 *arXiv:2305.13245*，2023年。

[2] Iz Beltagy、Matthew E Peters 和 Arman Cohan 所著的《Longformer：长文档变换器》。*arXiv*预印本 *arXiv:2004.05150*，2020年。

[3] Beidi Chen, Tri Dao, Eric Winsor, Zhao Song, Atri Rudra, and Christopher Ré. Scatterbrain: Unifying sparse and low-rank attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[4] Krzysztof Marcin Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Quincy Davis, Afroz Mohiuddin, Lukasz Kaiser, et al. Rethinking attention with performers. In *International Conference on Learning Representations (ICLR)*, 2020.

[5] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.

[6] Zhe Jia and Peter Van Sandt. Dissecting the Ampere GPU architecture via microbenchmarking. GPU Technology Conference, 2021.

[7] Zhe Jia, Marco Maggioni, Benjamin Staiger, and Daniele P Scarpazza. Dissecting the nvidia Volta GPU architecture via microbenchmarking. *arXiv preprint arXiv:1804.06826*, 2018.

[8] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning*, pages 5156–5165. PMLR, 2020.

[9] Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *The International Conference on Machine Learning (ICML)*, 2020.

[10] Benjamin Lefaudeux, Francisco Massa, Diana Liskovich, Wenhan Xiong, Vittorio Caggiano, Sean Naren, Min Xu, Jieru Hu, Marta Tintore, Susan Zhang, Patrick Labatut, and Daniel Haziza. xformers: A modular and hackable transformer modelling library. `https://github.com/facebookresearch/xformers`, 2022.

[11] Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax. *arXiv preprint arXiv:1805.02867*, 2018.

[12] OpenAI. Gpt-4 technical report. *ArXiv*, abs/2303.08774, 2023.

[13] Markus N Rabe and Charles Staats. Self-attention does not need $O(n^2)$ memory. *arXiv preprint arXiv:2112.05682*, 2021.

[14] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics*, 9: 53–68, 2021.

[15] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

[16] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

[17] Philippe Tillet, Hsiang-Tsung Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pages 10–19, 2019.

[18] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[19] Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.

[20] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. Big bird: Transformers for longer sequences. *Advances in Neural Information Processing Systems*, 33, 2020.

[3] Beidi Chen, Tri Dao, Eric Winsor, Zhao Song, Atri Rudra, and Christopher Ré. Scatterbrain: Unifying sparse and low-rank attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[4] Krzysztof Marcin Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Quincy Davis, Afroz Mohiuddin, Lukasz Kaiser, et al. Rethinking attention with performers. In *International Conference on Learning Representations (ICLR)*, 2020.

[5] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.

[6] Zhe Jia and Peter Van Sandt. Dissecting the Ampere GPU architecture via microbenchmarking. GPU Technology Conference, 2021.

[7] Zhe Jia, Marco Maggioni, Benjamin Staiger, and Daniele P Scarpazza. Dissecting the nvidia Volta GPU architecture via microbenchmarking. *arXiv preprint arXiv:1804.06826*, 2018.

[8] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning*, pages 5156–5165. PMLR, 2020.

[9] Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *The International Conference on Machine Learning (ICML)*, 2020.

[10] Benjamin Lefaudeux, Francisco Massa, Diana Liskovich, Wenhan Xiong, Vittorio Caggiano, Sean Naren, Min Xu, Jieru Hu, Marta Tintore, Susan Zhang, Patrick Labatut, and Daniel Haziza. xformers: A modular and hackable transformer modelling library. `https://github.com/facebookresearch/xformers`, 2022.

[11] Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax. *arXiv preprint arXiv:1805.02867*, 2018.

[12] OpenAI. Gpt-4 technical report. *ArXiv*, abs/2303.08774, 2023.

[13] Markus N Rabe and Charles Staats. Self-attention does not need $O(n^2)$ memory. *arXiv preprint arXiv:2112.05682*, 2021.

[14] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics*, 9: 53–68, 2021.

[15] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

[16] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

[17] Philippe Tillet, Hsiang-Tsung Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pages 10–19, 2019.

[18] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[19] Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.

[20] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. Big bird: Transformers for longer sequences. *Advances in Neural Information Processing Systems*, 33, 2020.